

Programare funcțională

Introducere în programarea funcțională folosind Haskell

C01 - Organizare. Intro în Haskell

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Welcome



Organizare

- Curs

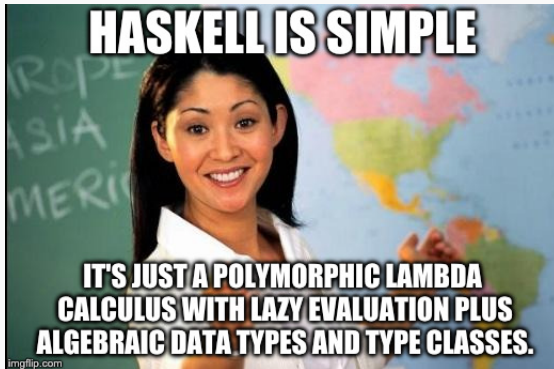
- Claudia Chiriță (webpage) (seria 23)
- Denisa Diaconescu (webpage) (seriile 24, 25)

- Laborator

- Ioan Ionescu (241, 242)
- Ștefan Mocanu (233, 234)
- Alex Oltean (232, 251, 252)
- Stelian Stoian (231)
- Dafina Truș (243, 244)

- Materiale pentru curs și laborator:
<https://tinyurl.com/PF2025-git>
- Canal Teams:
<https://tinyurl.com/PF2025-MTeams>

Prezența la curs nu este obligatorie, dar încurajată.



Notare

- **Sesiune:** 1 (oficiu) + notă laborator + examen
- **Restanță 1:** la alegere una din variantele de mai jos
 - 1 (oficiu) + notă laborator + examen (dacă aveți notă la laborator și vreți să o păstrați)
 - 1 (oficiu) + examen (altfel)
- **Restanță 2:** 1 (oficiu) + examen

Condiție de promovabilitate

- **cel puțin 5** > 4.99

Notă laborator

- max. 2 puncte
- se acordă pentru exercițiile rezolvate în cadrul laboratorului

Bonus

- max. 1 punct
- se acordă pentru activitate excepțională (de ex., rezolvarea exercițiilor suplimentare)

- max. 7 puncte
- durata 2 ore
- în sesiune, fizic
- 28 întrebări grilă
- un cheatsheet comun
 - Concurs "Best Cheatsheet"
 - mai multe detalii în decembrie

- Pagina Haskell
<http://haskell.org>
- Hoogle
<https://www.haskell.org/hoogle>
- Haskell Wiki
<http://wiki.haskell.org>
- Cartea online „Learn You a Haskell for Great Good”
<http://learnyouahaskell.com/>

De ce programare funcțională?

Ce știți despre programarea funcțională?



<https://tinyurl.com/PF-C01-Quiz1>

Programare Imperativă vs. Declarativă

Imperativă (*the How*)

- Îi dai calculatorului o listă clară de pași.
- Gândește-te la un **checklist** sau la o **rețetă**: faci asta, apoi asta, apoi asta.
- Programare procedurală / orientată pe obiecte.

Declarativă (*the What*)

- Îi spui doar ce vrei să obții, nu fiecare pas de urmat.
- Ca la restaurant: tu spui doar "vreau paste"; bucătarul decide cum gătește pastele.
- Programare logică / baze de date / funcțională

Programare funcțională în limbajul vostru preferat
(Java 8, C++11, Python, JavaScript, ...)

- Funcții anonime
- Funcții de procesare a fluxurilor de date: filter, map, reduce

Agregarea datelor dintr-o colecție (JS)

C. Boesch, Declarative vs Imperative Programming - Talk.JS

<https://www.youtube.com/watch?v=M2e5sq1rnvc>

Will I ever prefer to read declarative javascript?

```
function multiply(array) {  
  return array.reduce( (a,b) => a*b, 1);  
}
```

```
function multiply(array) {  
  var total = 1;  
  for (var i = 0; i < array.length; i++){  
    total = total * array[i];  
  }  
  return total;  
}
```

C. Boesch, Declarative vs Imperative Programming - Talk.JS

<https://www.youtube.com/watch?v=M2e5sq1rnvc>

Reasons to be More Declarative

- Better readability
- Better scalability
- Fewer state-related bugs
- Stand on the shoulders of giants



De ce Haskell?



Programare funcțională pură

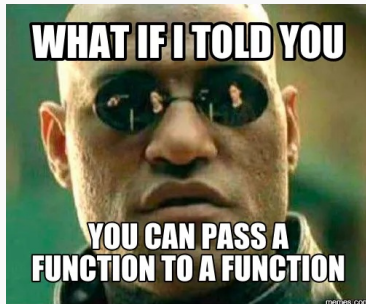
- Totul este o funcție
- O funcție produce mereu același rezultat pentru aceeași intrare
- O funcție nu poate modifica intrarea și nu poate produce *side effects* (e.g., citire dintr-un fișier, afișarea unui text)



De ce Haskell?

Funcțiile sunt *first-class citizens*

- Funcțiile sunt folosite ca valori
- Funcțiile pot fi transmise ca argumente și pot fi returnate de alte funcții



De ce Haskell?

Lazy

- Amânarea calculelor
 - orice expresie este evaluată doar atunci când chiar ai nevoie de rezultat
- Schimbă modul de gândire
 - programele nu mai sunt doar o succesiune de pași executați imediat, ci devin un set de descrieri pe care mașina le "activează" la momentul potrivit
- Colecții infinite
 - poți lucra elegant cu structuri de date infinite, de exemplu lista [1..], fără să explodeze programul
 - poți decide explicit când și cât dintr-o structură de date să fie evaluat
- Optimizare automată
 - evaluarea lazy evită calcule inutile, ceea ce poate reduce timpul de execuție

De ce Haskell?

Strongly Typed

- Orice are un tip
- Compilatorul verifică tipurile la compile-time
- Compilatorul deduce tipurile



De ce Haskell?

Limbaj elegant

- **Concepte puternice din matematică**
 - recursivitate, compunerea de funcții, functori, monade
 - λ -calcul
 - **Rigurozitate**
 - gândim mai mult înainte și producem cod mai curat
 - **Curbă de învățare în trepte**
 - putem scrie programe mici destul de repede
 - expertiza în Haskell necesită multă gândire și mult exercițiu
 - descoperirea unei lumi noi poate fi fun
- <http://wiki.haskell.org/Humor>

10 REASONS TO USE HASKELL



MEMORY SAFETY



GARBAGE COLLECTION



NATIVE CODE



STATIC TYPES



RICH TYPES



PURITY



LAZINESS



CONCURRENCY



METAPROGRAMMING



ECOSYSTEM

@serokell

@impurepics

<https://serokell.io/blog/10-reasons-to-use-haskell>

Proiecte din industrie în Haskell

- **Compilerul GHC pentru Haskell**
- **pandoc**
 - convertor de documente versatil, care suportă multe formate de fișiere.
- **Xmonad**
 - *tiling window manager* dinamic pentru sistemul X Window.
- **PostgREST**
 - server web care expune o bază de date PostgreSQL printr-un API RESTful.
- **Cardano**
 - blockchain

Programare funcțională în Haskell

- Tipuri, funcții
- Recursivitate, liste
- Funcții de nivel înalt
- Polimorfism
- Tipuri de date algebrice
- Clase de tipuri
- Functori
- Monade

Exemplu 1

```
qsort :: Ord a => [a] -> [a]
qsort []      = []
qsort (p:xs) =
    (qsort lesser) ++ [p] ++ (qsort greater)
    where
        lesser  = filter (< p) xs
        greater = filter (>= p) xs
```

Exemplu 2

```
repMax :: Ord a => [a] -> [a]
repMax [] = []
repMax [x] = [x]
repMax (x:y:xs) = max m r : (r:rs)
  where r:rs = repMax (m : xs)
        m = max x y
```

Magia din spate: λ -calcul

În 1929-1932, Alonzo Church a propus λ -calculul ca sistem formal pentru logica matematică. În 1935 a argumentat că orice funcție calculabilă peste numere naturale poate fi calculată în λ -calcul.

$t =$ x (variabilă)
 $| \lambda x. t$ (abstractizare)
 $| t t$ (aplicare)



- Independent, în 1935, Alan Turing a introdus **mașina Turing**.
- În 1936, Turing a argumentat că orice funcție calculabilă peste numere naturale poate fi calculată de o mașină Turing.
- Turing a arătat echivalența celor două modele de calcul.
- Această echivalență a constituit o indicație puternică asupra "universalității" celor două modele (**Teza Church-Turing**).

Elemente de bază. Primii pași

Comentarii

```
-- comentariu pe o linie  
{- comentariu pe  
    mai multe  
    linii -}
```

Identificatori

- șiruri formate din litere, cifre, caracterele `_` și `'` (apostrof)
- identificatorii pentru variabile încep cu literă mică sau `_`
- identificatorii pentru tipuri și constructori încep cu majuscule
- Haskell este case sensitive

```
double x = 2 * x  
data Point a = Pt a a
```

Blocuri și indentare

Blocurile sunt delimitate prin indentare.

```
fact n = if n == 0
        then 1
        else n * fact (n-1)
```

```
trei = let
      a = 1
      b = 2
      in a + b
```

Echivalent, putem scrie

```
trei = let {a = 1; b = 2} in a + b
trei = let a = 1; b = 2 in a + b
```

Presupunem că fișierul `test.hs` conține

`x = 1`

`x = 2`

Ce valoare are `x`?

Variabile

Presupunem că fișierul `test.hs` conține

```
x = 1
```

```
x = 2
```

Ce valoare are `x`?

```
Prelude> :l test.hs
```

```
test.hs:2:1: error:
```

```
  Multiple declarations of 'x'
```

```
  Declared at: test.hs:1:1
```

```
               test.hs:2:1
```

```
2 | x=2
```

```
  ^
```

În Haskell, variabilele sunt **imutabile** (*immutable*), adică:

- `=` **nu** este operator de atribuire
- `x = 1` reprezintă o **legătură** (*binding*)
- din momentul în care o variabilă este legată la o valoare, acea valoare nu mai poate fi schimbată

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

Presupunem că fișierul `testlet.hs` conține

```
x = 1
```

```
z = let x = 3 in x
```

Ce valoare au `z` și `x`?

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

Presupunem că fișierul `testlet.hs` conține

```
x = 1  
z = let x = 3 in x
```

Ce valoare au `z` și `x`?

```
-- z = 3  
-- x = 1
```

Legarea variabilelor

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

```
x = let
    z = 5
    g u = z + u
  in let
    z = 7
    in g 0 + z
```

Ce valoare are x?

Legarea variabilelor

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

```
x = let
    z = 5
    g u = z + u
  in let
    z = 7
    in g 0 + z
```

Ce valoare are x?

```
-- x = 12
```

Legarea variabilelor

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

```
x = let
    z = 5
    g u = z + u
in let
    z = 7
    in g 0 + z
```

Ce valoare are x?

```
-- x = 12
```

Dar în situația de mai jos, ce valoare are x?

```
x = let z = 5; g u = z + u in let z = 7 in g 0
```

Legarea variabilelor

let .. in ... este o **expresie** care creează un domeniu de vizibilitate local.

```
x = let
    z = 5
    g u = z + u
  in let
    z = 7
    in g 0 + z
```

Ce valoare are x?

```
-- x = 12
```

Dar în situația de mai jos, ce valoare are x?

```
x = let z = 5; g u = z + u in let z = 7 in g 0
-- x = 5
```


clauza **... where ...** creează un domeniu de vizibilitate local

```
f x = g x + g x + z
```

```
  where
```

```
    g x = 2*x
```

```
    z = x-1
```

Ce valoare are f 1?

clauza **... where ...** creează un domeniu de vizibilitate local

```
f x = g x + g x + z
  where
    g x = 2*x
    z = x-1
```

Ce valoare are f 1?

```
-- x = 4
```

Legarea variabilelor

let .. in ... este o expresie

`x = [let y = 8 in y, 9]` `-- x = [8,9]`

where este o clauză, disponibilă doar la nivel de definiție

`x = [y where y = 8, 9] - error: parse error ...`

Legarea variabilelor

let .. in ... este o expresie

```
x = [let y = 8 in y, 9]          -- x = [8,9]
```

where este o clauză, disponibilă doar la nivel de definiție

```
x = [y where y = 8, 9] - error: parse error ...
```

Variabile pot fi legate și prin *pattern matching* la definirea unei funcții sau expresii **case**.

```
h x | x == 0    = 0
    | x == 1    = y + 1
    | x == 2    = y * y
    | otherwise = y
where y = x*x
```

```
f x = case x of
        0 -> 0
        1 -> y + 1
        2 -> y * y
        _ -> y
where y = x*x
```

Quiz time!



<https://tinyurl.com/PF-C01-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C02 - Tipuri de date

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Tipuri de date

Tipuri de date. Sistemul tipurilor

"There are three interesting aspects to types in Haskell: they are strong, they are static, and they can be automatically inferred."

[http://book.realworldhaskell.org/read/
types-and-functions.html](http://book.realworldhaskell.org/read/types-and-functions.html)

strong – garantează absența anumitor erori

static – tipul fiecărei valori este calculat la compilare

dedus automat – compilatorul deduce automat tipul fiecărei expresii

```
Prelude> :t [('a',1,"abc")]  
[('a',1,"abc")] :: Num b => [(Char, b, [Char])]
```

Tipurile de bază: Int, Integer, Float, Double, Bool, Char, String

Tipurile de bază: Int, Integer, Float, Double, Bool, Char, String

Tipuri compuse: tupluri și liste

```
Prelude> :t ('a', True)
('a', True) :: (Char, Bool)
```

```
Prelude> :t ["Tom", "Jerry"]
["Tom", "Jerry"] :: [[Char]]
```

Sistemul tipurilor

Tipurile de bază: Int, Integer, Float, Double, Bool, Char, String

Tipuri compuse: tupluri și liste

```
Prelude> :t ('a', True)
('a', True) :: (Char, Bool)
```

```
Prelude> :t ["Tom", "Jerry"]
["Tom", "Jerry"] :: [[Char]]
```

Tipuri noi definite:

```
data RGB = Red | Green | Blue
data Point a = Pt a a      -- tip parametrizat
                           -- a este variabila de tip
```

Tipuri de date

Integer: 4, 0, -5

```
Prelude> 4 + 3
```

```
Prelude> (+) 4 3
```

```
Prelude> mod 4 3
```

```
Prelude> 4 'mod' 3
```

Float: 3.14

```
Prelude> truncate 3.14
```

```
Prelude> sqrt 4
```

```
Prelude> let x = 4 :: Int
```

```
Prelude> sqrt (fromIntegral x)
```

Char: 'a','A','\n'

```
Prelude> import Data.Char
```

```
Prelude Data.Char> chr 65
```

```
Prelude Data.Char> ord 'A'
```

```
Prelude Data.Char> toUpper 'a'
```

```
Prelude Data.Char> digitToInt '4'
```

Tipuri de date

Bool: True, False

```
data Bool = True | False
```

```
Prelude> True && False || True
```

```
Prelude> not True
```

```
Prelude> 1 /= 2
```

```
Prelude> 1 == 2
```

String: "Tom and Jerry"

```
type String = [Char] -- sinonim pentru tip
```

```
Prelude> "Tom"++"Jerry"  
"TomJerry"
```

```
Prelude> "TomJerry" !! 2  
'm'
```

```
Prelude> lines "Tom\nJerry"  
["Tom","Jerry"]
```

```
Prelude> words "Tom_and_Jerry"  
["Tom","and","Jerry"]
```

Tipul listă

```
Prelude>:t [True, False, True]  
[True, False, True] :: [Bool]
```

Tipul listă

```
Prelude>:t [True, False, True]  
[True, False, True] :: [Bool]
```

Tipul tuplu

```
Prelude> :t (1 :: Int, 'a', "ab")  
(1 :: Int, 'a', "ab") :: (Int, Char, [Char])
```

```
Prelude> fst (1, 'a') -- doar pentru perechi
```

```
Prelude> snd (1, 'a') -- doar pentru perechi
```


Tipuri. Clase de tipuri. Variabile de tip

Ce răspuns primim?

```
Prelude> :t 1
```

Tipuri. Clase de tipuri. Variabile de tip

Ce răspuns primim?

```
Prelude> :t 1
```

Răspunsul este:

```
1 :: Num a => a
```

Semnificația este următoarea:

- Num este o clasă de tipuri
- a este un parametru de tip
- 1 este o valoare de tipul a din clasa Num

Tipuri. Clase de tipuri. Variabile de tip

Ce răspuns primim?

```
Prelude> :t 1
```

Răspunsul este:

```
1 :: Num a => a
```

Semnificația este următoarea:

- Num este o clasă de tipuri
- a este un parametru de tip
- 1 este o valoare de tipul a din clasa Num

```
Prelude> :t [1,2,3]
```

```
[1,2,3] :: Num t => [t]
```

Erori de tip

Eroarea pe care o să o întâlniți cel mai des:

Couldn't match expected type

```
[Prelude> "string" ++ True  
  
<interactive>:1:13: error:  
  • Couldn't match expected type '[Char]' with actual type 'Bool'  
  • In the second argument of '(++)', namely 'True'  
    In the expression: "string" ++ True  
    In an equation for 'it': it = "string" ++ True
```

Chiar dacă pare lungă și scary, lucrurile nu stau chiar așa.

Don't panic: citiți eroarea.

Funcții

Prototipul funcției

- numele funcției
- semnatura funcției

double :: Integer -> Integer

Definiția funcției

- numele funcției
- parametrul formal
- corpul funcției

double **elem** = elem + elem

Aplicarea funcției

- numele funcției
- parametrul actual (argumentul)

double **5**

Exemplu: funcție cu două argumente

Prototipul funcției

add :: Integer -> Integer -> Integer

- numele funcției
- semnătura funcției

Definiția funcției

add elem1 elem2 = elem1 + elem2

- numele funcției
- parametrii formali
- corpul funcției

Aplicarea funcției

add 3 7

- numele funcției
- argumentele

Exemplu: funcție cu un argument de tip tuplu

Prototipul funcției

dist :: (Integer, Integer) -> Integer

- numele funcției
- semnatura funcției

Definiția funcției

dist (elem1, elem2) = abs (elem1 - elem2)

- numele funcției
- parametrul formal
- corpul funcției

Aplicarea funcției

dist (5, 7)

- numele funcției
- argumentul

Exemple de tipuri de funcții

Prelude> :t abs

abs :: Num a => a -> a

Prelude> :t (:)

(:) :: a -> [a] -> [a]

Prelude> :t (++)

(++) :: [a] -> [a] -> [a]

Prelude> :t zip

zip :: [a] -> [b] -> [(a, b)]

Definirea funcțiilor

```
fact :: Integer -> Integer
```

- Definiție folosind **if**

```
fact n = if n == 0 then 1
         else n * fact(n-1)
```

- Definiție folosind **pattern matching**

```
fact 0 = 1
fact n = n * fact(n-1)
```

- Definiție folosind **guards**

```
fact n
  | n == 0      = 1
  | otherwise = n * fact(n-1)
```

Pattern matching

```
fact :: Integer -> Integer
fact 0 = 1
fact n = n * fact(n-1)
```

- variabilele și valorile din partea stângă a lui = sunt *patterns*
- când funcția este apelată se încearcă potrivirea parametrilor actuali, ecuațiile fiind încercate în ordinea scrierii
- în definiția factorialului, 0 și n sunt patterns:
0 se va potrivi numai cu el însuși,
iar n se va potrivi cu orice valoare de tip Integer

Ordinea ecuațiilor este importantă!

Să presupunem că schimbăm ordinea ecuațiilor din definiția factorialului:

```
fact :: Integer -> Integer
fact n = n * fact(n-1)
fact 0 = 1
```

Ce se întâmplă?

Ordinea ecuațiilor este importantă!

Să presupunem că schimbăm ordinea ecuațiilor din definiția factorialului:

```
fact :: Integer -> Integer
fact n = n * fact(n-1)
fact 0 = 1
```

Ce se întâmplă?

Deoarece n este un pattern care se potrivește cu orice valoare, inclusiv cu 0, orice apel al funcției va alege prima ecuație.

Astfel, funcția **nu își va încheia execuția**.

Pattern matching

Tipul `Bool` este definit în Haskell astfel:

```
data Bool = True | False
```

Putem defini operația `||` astfel

```
(||) :: Bool -> Bool -> Bool
```

```
False || x = x
```

```
True  || _ = True
```

În acest exemplu `_`, `x`, `True` și `False` sunt patterns.

Observăm că `True` și `False` sunt constructori de date și se vor potrivi numai cu ei înșiși.

Pattern-ul `_` se numește *wild-card pattern*; el se potrivește cu orice valoare.

Tipuri de funcții

Fie `foo` o funcție cu următorul tip

`foo :: a -> b -> [a] -> [b]`

- are trei argumente, de tipuri `a`, `b` și `[a]`
- întoarce un rezultat de tip `[b]`

Tipuri de funcții

Fie `foo` o funcție cu următorul tip

```
foo :: a -> b -> [a] -> [b]
```

- are trei argumente, de tipuri `a`, `b` și `[a]`
- întoarce un rezultat de tip `[b]`

Schimbăm semnatura funcției astfel:

```
ffoo :: (a -> b) -> [a] -> [b]
```

- are două argumente, de tipuri `(a -> b)` și `[a]`,
adică o funcție de la `a` la `b` și o listă de elemente de tip `a`
- întoarce un rezultat de tip `[b]`

Prelude> :t map

```
map :: (a -> b) -> [a] -> [b]
```


Quiz time!



<https://tinyurl.com/PF-C02-Quiz1>

Liste

Orice listă poate fi scrisă folosind doar constructorul `(:)` și lista vidă `[]`.

- $[1,2,3] == 1 : (2 : (3 : [])) == 1 : 2 : 3 : []$
- $"abc" == ['a','b','c'] == 'a' : ('b' : ('c' : [])) == 'a' : 'b' : 'c' : []$

Definiție recursivă. O **listă** este

- **vidă**, notată `[]`, sau
- **compusă**, notată $x : xs$, dintr-un element x numit **capul listei** (*head*) și o listă xs numită **coada listei** (*tail*).

Definirea listelor. Operații

Intervale și progresii

```
interval = ['c'..'e']      -- ['c','d','e']
progresie = [20,17..1]     -- [20,17,14,11,8,5,2]
progresie' = [2.0,2.5..4.0] -- [2.0,2.5,3.0,3.5,4.0]
```

Operații

```
Prelude> import Data.List
Prelude> [1,2,3] !! 2
3
Prelude> "abcd" !! 0
'a'
Prelude> [1,2] ++ [3]
[1,2,3]
```

Definiția prin selecție $\{x \mid P(x)\}$

$[E(x) \mid x \leftarrow [x_1, \dots, x_n], P(x)]$

```
Prelude> xs = [0..10]
```

```
Prelude> [x | x <- xs, even x]  
[0,2,4,6,8,10]
```

```
Prelude> xs = [0..6]
```

```
Prelude> [(x,y) | x <- xs, y <- xs, x + y == 10]  
[(4,6),(5,5),(6,4)]
```

Definiția prin selecție $\{x \mid P(x)\}$

$$[E(x) \mid x \leftarrow [x_1, \dots, x_n], P(x)]$$

Putem folosi **let** pentru domeniu de vizibilitate local.

```
Prelude> [(i,j) | i <- [1..2],  
                let k = 2 * i, j <- [1..k]]  
[(1,1),(1,2),  
 (2,1),(2,2),(2,3),(2,4)]
```

```
Prelude> :t zip
```

```
zip :: [a] -> [b] -> [(a, b)]
```

```
Prelude> ys = ['A'..'E']
```

```
Prelude> zip [1..] ys
```

```
[(1,'A'),(2,'B'),(3,'C'),(4,'D'),(5,'E')]
```

```
Prelude> xs = ['A'..'Z']
```

```
Prelude> [x | (i,x) <- [1..] 'zip' xs, even i]
```

```
"BDFHJLNPRTVXZ"
```

Observați diferența!

```
Prelude> zip [1..3] ['A'..'D']  
[(1,'A'),(2,'B'),(3,'C')]
```

```
Prelude> [(x,y) | x <- [1..3], y <- ['A'..'D']]  
[(1,'A'),(1,'B'),(1,'C'),(1,'D'),  
  (2,'A'),(2,'B'),(2,'C'),(2,'D'),  
  (3,'A'),(3,'B'),(3,'C'),(3,'D')]
```


Lazy: argumentele sunt evaluate doar când e necesar și doar cât e necesar.

```
Prelude> head[]  
*** Exception: Prelude.head: empty list  
Prelude> x = head []  
Prelude> f a = 5  
Prelude> f x  
5  
Prelude> [1,head [],3] !! 0  
1  
Prelude> [1, head [],3] !! 1  
*** Exception: Prelude.head: empty list
```

Liste infinite

Drept consecință a **evaluării lazy**, se pot defini liste infinite.

```
Prelude> natural = [0..]
```

```
Prelude> take 5 natural
```

```
[0,1,2,3,4]
```

```
Prelude> evenNat = [0,2..] -- progresie infinita
```

```
Prelude> take 7 evenNat
```

```
[0,2,4,6,8,10,12]
```

```
Prelude> ones = [1,1..]
```

```
Prelude> zeros = [0,0..]
```

```
Prelude> both = zip ones zeros
```

```
Prelude> take 5 both
```

```
[(1,0),(1,0),(1,0),(1,0),(1,0)]
```

Șabloane (patterns) pentru liste

Listele sunt construite folosind constructorii (:) și []

`[1,2,3] == 1:[2,3] == 1:2:[3] == 1:2:3:[]`

```
Prelude> x:y = [1,2,3]
```

```
Prelude> x
```

```
1
```

```
Prelude> y
```

```
[2,3]
```

Ce s-a întâmplat?

- `x : y` este un pattern pentru liste
- potrivirea dintre `x : y` și `[1,2,3]` a avut ca efect:
 - "deconstrucția" valorii `[1,2,3]` în `1 : [2,3]`
 - legarea lui `x` la `1` și a lui `y` la `[2,3]`

Șabloane (patterns) pentru liste

Definiții folosind șabloane

```
reverse [] = []
```

```
reverse (x:xs) = (reverse xs) ++ [x]
```

x : xs se potrivește cu liste nevide.

Șabloanele sunt definite folosind constructori.

De exemplu, operația de concatenare pe liste este

$$(++) :: [a] \rightarrow [a] \rightarrow [a],$$

dar `[x] ++ [1] = [2,1]` **nu** va avea ca efect legarea lui `x` la `2`.

```
Prelude> [x] ++ [1] = [2,1]
```

```
Prelude> x
```

```
error: ...
```

Șabloanele sunt liniare

Șabloanele sunt **liniare**, adică o variabilă apare cel mult dată.

Șabloanele în care o variabilă apare de mai multe ori generează mesaje de eroare. De exemplu:

- `x:x:[1] = [2,2,1]`
- `ttail (x:x:t) = t`
- `foo x x = x^2`

error: Conflicting definitions for x

O soluție este folosirea gărzilor:

```
ttail (x:y:t) | (x==y) = t
              | otherwise = ...
```

```
foo x y | (x == y) = x^2
        | otherwise = ...
```

Operatori

Operatorii sunt funcții cu două argumente

Operatorii în Haskell

- au două argumente
- pot fi apelați folosind notația infix
- pot fi definiți folosind numai "simboluri" (ex: `*!*`)
 - în definiția tipului operatorul este scris între paranteze

- Operatori predefiniți

```
(||) :: Bool -> Bool -> Bool
```

```
(:) :: a -> [a] -> [a]
```

```
(+) :: Num a => a -> a -> a
```

- Operatori noi

```
(&&&) :: Bool -> Bool -> Bool -- atentie la paranteze
```

```
True &&& b = b
```

```
False &&& _ = False
```

Funcții ca operatori

Operatorii care sunt definiți în formă infix, sunt apelați în formă prefix folosind paranteze

`2 + 3 == (+) 2 3`

Operatorii care sunt definiți în formă prefix, sunt apelați în formă infix folosind `` (*backtick*)

`mod 5 2 == 5 `mod` 2`

Prelude> mod 5 2

1

Prelude> 5 `mod` 2

1

elem :: a -> [a] -> Bool

Prelude> 1 `elem` [1,2,3]

True

Precedență și asociativitate

```
Prelude> 3+5*4:[6]++8-2+3:[2]==[23,6,9,2]||True==False
True
```

Precedence	Left associative	Non-associative	Right associative
9	!!		.
8			^, ^^, **
7	*, /, 'div', 'mod', 'rem', 'quot'		
6	+, -		
5			:, ++
4		==, /=, <, <=, >, >=, 'elem', 'notElem'	
3			&&
2			
1	>>, >>=		
0			\$, \$!, 'seq'

Operatorul - asociativ la stânga

$5 - 2 - 1 == (5 - 2) - 1$

$-- /= 5 - (2 - 1)$

Operatorul : asociativ la dreapta

$5 : 2 : [] == 5 : (2 : [])$

Operatorul ++ asociativ la dreapta

$(++) :: [a] \rightarrow [a] \rightarrow [a]$

$[] ++ ys = ys$

$(x:xs) ++ ys = x:(xs ++ ys)$

$l1 ++ l2 ++ l3 ++ l4 ++ l5 == l1 ++ (l2 ++ (l3 ++ (l4 ++ l5)))$

Quiz time!



<https://tinyurl.com/PF-C02-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C03 - Funcții & Liste (continuare)

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Mai multe despre funcții

Funcții anonime

Funcțiile sunt valori (*first-class citizens*).

Funcțiile pot fi folosite ca argumente pentru alte funcții.

Funcții anonime = lambda expresii

\x1 x2 ... xn -> expresie

```
Prelude> (\x -> x + 1) 3
```

```
4
```

```
Prelude> inc = \x -> x + 1
```

```
Prelude> add = \x y -> x + y
```

```
Prelude> aplic = \f x -> f x
```

```
Prelude> map (\x -> x+1) [1,2,3,4]  
[2,3,4,5]
```

Funcțiile sunt valori

Exemplu:

flip :: (a -> b -> c) -> (b -> a -> c)

- definiția folosind șabloane

flip f x y = f y x

- definiția cu lambda expresii

flip f = \x y -> f y x

- flip** ca valoare de tip funcție

flip = \f x y -> f y x

Currying

Currying este procedeul prin care o funcție cu mai multe argumente este transformată într-o funcție care are un singur argument și întoarce o altă funcție.

- În Haskell toate funcțiile sunt în forma **curry**, deci au un singur argument.
- Operatorul $\rightarrow$ pe tipuri este asociativ la dreapta, așadar înțelegem tipul $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n$ ca $a_1 \rightarrow (a_2 \rightarrow \dots (a_{n-1} \rightarrow a_n) \dots)$.
- Aplicarea funcțiilor este asociativă la stânga, așadar expresia $f\ x_1 \dots x_n$ trebuie înțeleasă ca $(\dots ((f\ x_1)\ x_2) \dots x_n)$.

Funcțiile curry/uncurry și mulțimi

```
Prelude> :t curry
```

```
curry :: ((a, b) -> c) -> a -> b -> c
```

```
Prelude> :t uncurry
```

```
uncurry :: (a -> b -> c) -> (a, b) -> c
```

Exemplu:

```
f :: (Int, String) -> String
```

```
f (n,s) = take n s
```

```
Prelude> let cf = curry f
```

```
Prelude> :t cf
```

```
cf :: Int -> String -> String
```

```
Prelude> f(1,"abc")
```

```
"a"
```

```
Prelude> cf 1 "abc"
```

```
"a"
```

Matematic. Date fiind $f : A \rightarrow B$ și $g : B \rightarrow C$, compunerea lor, notată $g \circ f : A \rightarrow C$, este dată de formula

$$(g \circ f)(x) = g(f(x))$$

În Haskell.

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)  
(g . f) x = g (f x)
```

```
Prelude> :t reverse  
reverse :: [a] -> [a]
```

```
Prelude> :t take  
take :: Int -> [a] -> [a]
```

```
Prelude> :t take 5 . reverse  
take 5 . reverse :: [a] -> [a]
```

```
Prelude> (take 5 . reverse) [1..10]  
[10,9,8,7,6]
```

```
Prelude> (head . reverse . take 5) [1..10]  
5
```

Operatorul \$

Operatorul (\$) are precedența 0.

$(\$)\ ::\ (a \rightarrow b) \rightarrow a \rightarrow b$

$f\ \$\ x = f\ x$

```
Prelude> (head . reverse . take 5) [1..10]  
5
```

```
Prelude> head . reverse . take 5 $ [1..10]  
5
```

Operatorul (\$) este asociativ la dreapta.

```
Prelude> head $ reverse $ take 5 $ [1..10]  
5
```

Quiz time!



<https://tinyurl.com/PF-C03-Quiz1>

Procesarea fluxurilor de date: Map, Filter, Fold

Map:

transformarea fiecărui element dintr-o listă

Exemplu - Pătrate

Definiți o funcție care pentru o listă de numere întregi dată ridică la pătrat fiecare element din listă.

Exemplu - Pătrate

Definiți o funcție care pentru o listă de numere întregi dată ridică la pătrat fiecare element din listă.

Soluție recursivă

```
squares :: [Int] -> [Int]
squares []      = []
squares (x:xs) = x*x : squares xs
```

Exemplu - Pătrate

Definiți o funcție care pentru o listă de numere întregi dată ridică la pătrat fiecare element din listă.

Soluție recursivă

```
squares :: [Int] -> [Int]
squares []      = []
squares (x:xs) = x*x : squares xs
```

Soluție descriptivă

```
squares :: [Int] -> [Int]
squares xs = [ x * x | x <- xs ]
```

```
Prelude> squares [1,-2,3]
[1,4,9]
```

Exemplu - Coduri ASCII

Transformați un șir de caractere în lista codurilor ASCII ale caracterelor.

Exemplu - Coduri ASCII

Transformați un șir de caractere în lista codurilor ASCII ale caracterelor.

Soluție recursivă

```
ords :: [Char] -> [Int]
ords []      = []
ords (x:xs) = ord x : ords xs
```

Exemplu - Coduri ASCII

Transformați un șir de caractere în lista codurilor ASCII ale caracterelor.

Soluție recursivă

```
ords :: [Char] -> [Int]
ords []      = []
ords (x:xs) = ord x : ords xs
```

Soluție descriptivă

```
ords :: [Char] -> [Int]
ords xs = [ ord x | x <- xs ]
```

```
Prelude> ords "a2c3"
[97,50,99,51]
```

Funcția `map`

Date fiind o funcție de transformare și o listă,
aplicați funcția fiecărui element al unei liste date.

Soluție recursivă

```
map :: (a -> b) -> [a] -> [b]
map f []          = []
map f (x:xs)      = f x : map f xs
```

Soluție descriptivă

```
map :: (a -> b) -> [a] -> [b]
map f xs = [ f x | x <- xs ]
```

Exemplu — Pătrate

Soluție recursivă

```
squares :: [Int] -> [Int]
squares []      = []
squares (x:xs) = x*x : squares xs
```

Soluție descriptivă

```
squares :: [Int] -> [Int]
squares xs = [ x * x | x <- xs ]
```

Soluție folosind map

```
squares :: [Int] -> [Int]
squares xs = map sqr xs
              where sqr x = x * x
```


Exemplu — Coduri ASCII

Soluție recursivă

```
ords :: [Char] -> [Int]
ords []      = []
ords (x:xs) = ord x : ords xs
```

Soluție descriptivă

```
ords :: [Char] -> [Int]
ords xs = [ ord x | x <- xs ]
```

Soluție folosind map

```
ords :: [Char] -> [Int]
ords xs = map ord xs
```

Funcții de ordin înalt

```
map :: (a -> b) -> [a] -> [b]
map f xs = [f x | x <- xs]
```

```
Prelude> map ($ 3) [(4 +), (10 *), (^ 2), sqrt]
[7.0,30.0,9.0,1.7320508075688772]
```

În acest caz:

- primul argument este o secțiune a operatorului (\$) (transmitem doar o parte din parametri)
- al doilea argument este o listă de funcții

$$\text{map } (\$ x) [f_1, \dots, f_n] == [f_1 x, \dots, f_n x]$$

Filter:

selectarea elementelor dintr-o listă

Exemplu - Selectarea elementelor pozitive dintr-o listă

Definiți o funcție care selectează elementele pozitive dintr-o listă.

Exemplu - Selectarea elementelor pozitive dintr-o listă

Definiți o funcție care selectează elementele pozitive dintr-o listă.

Soluție recursivă

```
positives :: [Int] -> [Int]
positives [] = []
positives (x:xs) | x > 0 = x : positives xs
                  | otherwise = positives xs
```

Exemplu - Selectarea elementelor pozitive dintr-o listă

Definiți o funcție care selectează elementele pozitive dintr-o listă.

Soluție recursivă

```
positives :: [Int] -> [Int]
positives [] = []
positives (x:xs) | x > 0 = x : positives xs
                  | otherwise = positives xs
```

Soluție descriptivă

```
positives :: [Int] -> [Int]
positives xs = [ x | x <- xs, x > 0 ]
```

```
Prelude> positives [1,-2,3]
[1,3]
```

Exemplu - Selectarea cifrelor dintr-un șir de caractere

Definiți o funcție care selectează cifrele dintr-un șir de caractere.

Exemplu - Selectarea cifrelor dintr-un șir de caractere

Definiți o funcție care selectează cifrele dintr-un șir de caractere.

Soluție recursivă

```
digits :: [Char] -> [Char]
digits [] = []
digits (x:xs) | isDigit x = x : digits xs
               | otherwise = digits xs
```


Exemplu - Selectarea cifrelor dintr-un șir de caractere

Definiți o funcție care selectează cifrele dintr-un șir de caractere.

Soluție recursivă

```
digits :: [Char] -> [Char]
digits [] = []
digits (x:xs) | isDigit x = x : digits xs
               | otherwise = digits xs
```

Soluție descriptivă

```
digits :: [Char] -> [Char]
digits xs = [ x | x <- xs, isDigit x ]
```

```
Prelude> digits "a2c3"
"23"
```

Funcția filter

Date fiind un predicat (funcție booleană) și o listă, selectați elementele din listă care satisfac predicatul.

Soluție recursivă

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs) | p x = x : filter p xs
                  | otherwise = filter p xs
```

Soluție descriptivă

```
filter :: (a -> Bool) -> [a] -> [a]
filter p xs = [ x | x <- xs, p x ]
```

Exemplu — Selectarea elementelor pozitive dintr-o listă

Soluție recursivă

```
positives :: [Int] -> [Int]
positives [] = []
positives (x:xs) | x > 0 = x : positives xs
                  | otherwise = positives xs
```

Soluție descriptivă

```
positives :: [Int] -> [Int]
positives xs = [ x | x <- xs, x > 0 ]
```

Soluție folosind filter

```
positives :: [Int] -> [Int]
positives xs = filter pos xs
               where pos x = x > 0
```

Exemplu — Selectarea cifrelor dintr-un șir de caractere

Soluție recursivă

```
digits :: [Char] -> [Char]
digits [] = []
digits (x:xs) | isDigit x = x : digits xs
               | otherwise = digits xs
```

Soluție descriptivă

```
digits :: [Char] -> [Char]
digits xs = [ x | x <- xs, isDigit x ]
```

Soluție folosind filter

```
digits :: [Char] -> [Char]
digits xs = filter isDigit xs
```

Quiz time!



<https://tinyurl.com/PF-C03-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell

C04 - Fold

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Procesarea fluxurilor de date: Map, Filter, Fold

Fold:

agregarea elementelor dintr-o listă

Exemplu - Suma

Definiți o funcție care, dată fiind o listă de numere întregi, calculează suma elementelor din listă.

Soluție recursivă

```
sum :: [Int] -> Int
sum []      = 0
sum (x:xs) = x + sum xs
```

```
Prelude> sum [1,2,3,4]
10
```

Definiți o funcție care, dată fiind o listă de numere întregi, calculează produsul elementelor din listă.

Soluție recursivă

```
product :: [Int] -> Int  
product []      = 1  
product (x:xs) = x * product xs
```

```
Prelude> product [1,2,3,4]  
24
```

Exemplu - Concatenare

Definiți o funcție care concatenează o listă de liste.

Soluție recursivă

```
concat :: [[a]] -> [a]
concat []      = []
concat (xs:xss) = xs ++ concat xss
```

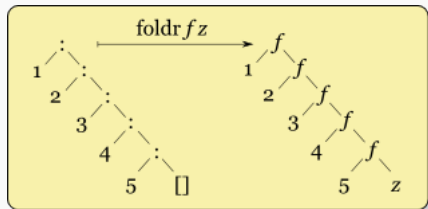
```
Prelude> concat [[1,2,3],[4,5]]
[1,2,3,4,5]
```

```
Prelude> concat ["Hello_", "world", "!"]
"Hello_world!"
```

Funcția foldr

foldr :: (a -> b -> b) -> b -> [a] -> b

Date fiind o funcție de actualizare a valorii calculate cu un element curent, o valoare inițială, și o listă, calculează valoarea obținută prin aplicarea repetată a funcției de actualizare fiecărui element din listă.



foldr :: (a -> b -> b) -> b -> [a] -> b

Soluție recursivă

foldr :: (a -> b -> b) -> b -> [a] -> b

foldr f i [] = i

foldr f i (x:xs) = f x (**foldr** f i xs)

Exemplu — Suma

Soluție recursivă

```
sum :: [Int] -> Int
sum []      = 0
sum (x:xs) = x + sum xs
```

Soluție folosind foldr

```
sum :: [Int] -> Int
sum xs = foldr (+) 0 xs
```

Exemplu

```
foldr (+) 0 [1, 2, 3] == 1 + (2 + (3 + 0))
```

Exemplu — Produs

Soluție recursivă

```
product :: [Int] -> Int
product []      = 1
product (x:xs) = x * product xs
```

Soluție folosind foldr

```
product :: [Int] -> Int
product xs = foldr (*) 1 xs
```

Exemplu

```
foldr (*) 1 [1, 2, 3] == 1 * (2 * (3 * 1))
```


Exemplu — Concatenare

Soluție recursivă

```
concat :: [[a]] -> [a]
concat []          = []
concat (xs:xss) = xs ++ concat xss
```

Soluție folosind foldr

```
concat :: [Int] -> Int
concat xs = foldr (++) [] xs
```

Exemplu

```
foldr (++) [] ["Hello_", "world", "!"]
  == "Hello_" ++ ("world_" ++ ("!" ++ []))
```

Exemplu – Suma pătratelor numerelor pozitive

Varianta 1:

```
f :: [Int] -> Int
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```

Exemplu – Suma pătratelor numerelor pozitive

Varianta 1:

```
f :: [Int] -> Int
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```

Varianta 2:

```
f :: [Int] -> Int
f xs = foldr (+) 0
      (map (\ x -> x * x)
        (filter (\ x -> x > 0) xs))
```

Exemplu – Suma pătratelor numerelor pozitive

Varianta 3:

```
f :: [Int] -> Int  
f xs = foldr (+) 0 (map ( ^ 2) (filter ( > 0) xs))
```

Exemplu – Suma pătratelor numerelor pozitive

Varianta 3:

```
f :: [Int] -> Int  
f xs = foldr (+) 0 (map ( ^ 2) (filter ( > 0) xs))
```

Varianta 4:

```
f :: [Int] -> Int  
f = foldr (+) 0 . map (^2) . filter (>0)
```

Exemplu - Compunerea funcțiilor

În definiția lui **foldr**, tipul **b** poate fi tipul unei funcții.

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

Putem defini compunerea funcțiilor dintr-o listă de funcții:

```
compose :: [a -> a] -> (a -> a)
```

```
compose = foldr (.) id
```

Funcția **id** este funcția identitate pentru orice tip:

```
id :: a -> a
```

```
Prelude> compose [(+1), (^2)] 3
```

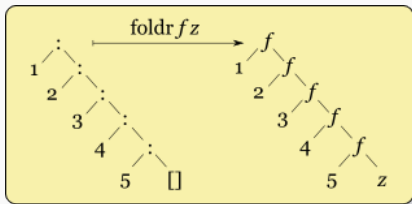
```
10
```

Quiz time!

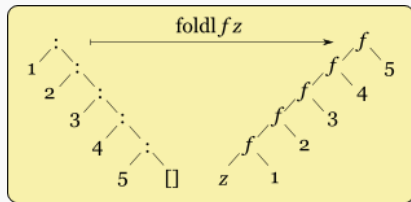
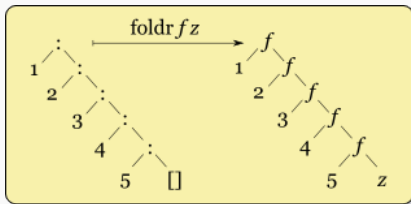


<https://tinyurl.com/PF-C04-Quiz1>

foldr și foldl



foldr și foldl



foldr și foldl

Date fiind o funcție de actualizare a valorii calculate cu un element curent, o valoare inițială, și o listă, calculează valoarea obținută prin aplicarea repetată a funcției de actualizare fiecărui element din listă.

```
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr op z [a1, a2, a3, ..., an] =  
    a1 `op` (a2 `op` (a3 `op` (...(an `op` z)...)))
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b  
foldl op z [a1, a2, a3, ..., an] =  
    (...(((z `op` a1) `op` a2) `op` a3) ...) `op` an
```

Funcția **foldr**

foldr :: (a -> b -> b) -> b -> [a] -> b

foldr f i [] = i

foldr f i (x:xs) = f x (**foldr** f i xs)

Funcția **foldl**

foldl :: (b -> a -> b) -> b -> [a] -> b

foldl h i [] = i

foldl h i (x:xs) = **foldl** h (h i x) xs

Suma elementelor dintr-o listă

Soluție cu foldr

```
sum :: [Int] -> Int  
sum = foldr (+) 0
```

Cu **foldr**, elementele sunt procesate de la dreapta la stânga:

$$\text{sum } [x_1, \dots, x_n] = (x_1 + (x_2 + \dots (x_n + 0) \dots))$$

Suma elementelor dintr-o listă

Soluție în care elementele sunt procesate de la stânga la dreapta:

```
sum :: [Int] -> Int
sum xs = suml xs 0
    where
        suml [] n = n
        suml (x:xs) n = suml xs (n + x)
```

Observați:

$$\text{suml } [x_1, \dots, x_n] 0 = (\dots (0 + x_1) + x_2) + \dots x_n$$

Suma elementelor dintr-o listă

Soluție în care elementele sunt procesate de la stânga la dreapta:

```
sum :: [Int] -> Int
sum xs = suml xs 0
    where
        suml [] n = n
        suml (x:xs) n = suml xs (n + x)
```

Observați:

$$\text{suml } [x_1, \dots, x_n] 0 = (\dots (0 + x_1) + x_2) + \dots x_n$$

Soluție cu foldl

```
sum :: [Int] -> Int
sum xs = foldl (+) 0 xs
```

Inversarea elementelor unei liste

Definiți o funcție care, dată fiind o listă de elemente, calculează lista în care elementele sunt scrise în ordine inversă.

Soluție cu foldl

```
rev = foldl (<:>) []  
      where (<:>) = flip (:)  
  
--      flip :: (a -> b -> c) -> (b -> a -> c)  
--      (:) :: a -> [a] -> [a]  
--      flip (:) :: [a] -> a -> [a]
```

Elementele sunt procesate de la stânga la dreapta:

rev $[x_1, \dots, x_n] = (\dots(([] <:> x_1) <:> x_2) \dots) <:> x_n$

Evaluare lazy. Liste infinite

Putem folosi funcțiile **map** și **filter** pe liste infinite:

```
Prelude> inf = map (+10) [1..] -- inf nu este evaluat
```

```
Prelude> take 3 inf
```

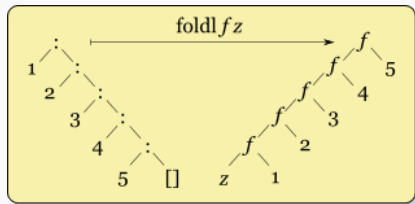
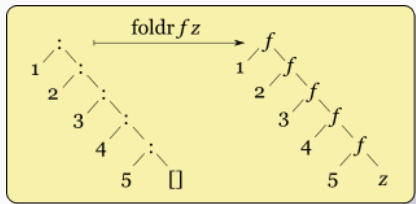
```
[11,12,13]
```

Lazy evaluation!

- expresiile sunt evaluate numai când este nevoie de valoarea lor
- expresiile nu sunt evaluate total, elementele care nu sunt folosite rămân neevaluate
- o expresie este evaluată o singură dată.

În exemplul de mai sus, este acceptată definiția lui `inf`, fără a fi evaluată. Când expresia `take 3 inf` este evaluată, numai primele 3 elemente ale lui `inf` sunt calculate, restul rămânând neevaluate.

foldr și foldl



- **foldr** poate fi folosită pe liste infinite (în anumite cazuri)
- **foldl** nu poate fi folosită pe liste infinite niciodată

foldr și foldl

- **foldr** poate fi folosită pe liste infinite (în anumite cazuri),
- **foldl** **nu** poate fi folosită pe liste infinite niciodată.

```
Prelude> foldr (*) 0 [1..]
```

```
*** Exception: stack overflow
```

```
Prelude> take 3 $ foldr (\x xs -> (x+1):xs) [] [1..]  
[2,3,4]
```

```
-- foldr a functionat pe o lista infinita
```

```
Prelude> take 3 $ foldl (\xs x -> (x+1):xs) [] [1..]
```

```
-- expresia se calculeaza la infinit
```

Quiz time!



<https://tinyurl.com/PF-C04-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C05

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Tipuri de date algebrice

Tipuri de date algebrice

Informal, un **tip de date algebric** este o combinație de

- **tipuri sumă** și
- **tipuri produs**

Tipuri sumă

În Haskell, tipul `Bool` este definit astfel:

```
data Bool = False | True
```

- `Bool` este constructor de tip
- `False` și `True` sunt constructori de date

Tipuri sumă

În Haskell, tipul `Bool` este definit astfel:

```
data Bool = False | True
```

- `Bool` este constructor de tip
- `False` și `True` sunt constructori de date

În mod similar, putem defini

```
data Season = Spring | Summer | Autumn | Winter
```

- `Season` este constructor de tip
- `Spring`, `Summer`, `Autumn` și `Winter` sunt constructori de date

Tipuri sumă

În Haskell, tipul **Bool** este definit astfel:

```
data Bool = False | True
```

- **Bool** este constructor de tip
- **False** și **True** sunt constructori de date

În mod similar, putem defini

```
data Season = Spring | Summer | Autumn | Winter
```

- **Season** este constructor de tip
- **Spring**, **Summer**, **Autumn** și **Winter** sunt constructori de date

Bool și **Season** sunt tipuri de date sumă, adică sunt definite prin enumerarea alternativelor.

Tipuri sumă: Bool

```
data Bool = False | True
```

Operațiile se definesc prin "pattern matching":

```
not :: Bool -> Bool
```

```
not False = True
```

```
not True  = False
```

```
(&&), (||) :: Bool -> Bool -> Bool
```

```
False && q = False
```

```
True  && q = q
```

```
False || q = q
```

```
True  || q = True
```

Tipuri sumă: Season

```
data Season = Spring | Summer | Autumn | Winter
```

```
succesor :: Season -> Season
```

```
succesor Spring = Summer
```

```
succesor Summer = Autumn
```

```
succesor Autumn = Winter
```

```
succesor Winter = Spring
```

```
showSeason :: Season -> String
```

```
showSeason Spring = "Primavara"
```

```
showSeason Summer = "Vara"
```

```
showSeason Autumn = "Toamna"
```

```
showSeason Winter = "Iarna"
```

Tipuri produs

Problemă. Să definim un tip de date care să aibă ca valori "puncte" cu două coordonate de tipuri oarecare.

```
data Point a b = Pt a b
```

- Point este **constructor de tip**
- Pt este **constructor de date**

Pentru a accesa componentele, definim proiecțiile:

```
pr1 :: Point a b -> a  
pr1 (Pt x _) = x
```

```
pr2 :: Point a b -> b  
pr2 (Pt _ y) = y
```

Point este un **tip de date produs**, definit prin **combinarea** tipurilor a și b.

Tipuri produs

```
data Point a b = Pt a b
```

```
Prelude> :t (Pt 1 "c")  
(Pt 1 "c") :: Num a => Point a [Char]
```

```
Prelude> :t Pt  
Pt :: a -> b -> Point a b  
-- constructorul de date este o funcție
```

```
Prelude> :t (Pt 1)  
(Pt 1) :: Num a => b -> Point a b
```

Se pot defini operații:

```
pointFlip :: Point a b -> Point b a  
pointFlip (Pt x y) = Pt y x
```

```
data [a] = [] | a : [a]
```

Constructorii listelor sunt [] și :

```
[] :: [a]
```

```
(:) :: a -> [a] -> [a]
```

Declarația listelor ca tip de date algebric:

```
data List a = Nil  
            | Cons a (List a)
```

- **List** este constructor de tip
- Nil și Cons sunt constructori de date

Se pot defini operații:

```
append :: List a -> List a -> List a  
append Nil ys          = ys  
append (Cons x xs) ys = Cons x (append xs ys)
```


Tipuri de date algebrice

Tipurile de date algebrice se definesc folosind "operațiile" sumă și produs.

Forma generală:

$$\begin{aligned} \text{data Typename} = & \text{Cons}_1 \ t_{11} \dots t_{1k_1} \\ & | \text{Cons}_2 \ t_{21} \dots t_{2k_2} \\ & | \dots \\ & | \text{Cons}_n \ t_{n1} \dots t_{nk_n} \end{aligned}$$

unde $k_1, \dots, k_n \geq 0$

- Se pot folosi tipuri sumă și tipuri produs.
- Se pot defini tipuri parametrizate.
- Se pot folosi definiții recursive.

Tipuri de date algebrice

Forma generală:

$$\begin{aligned} \text{data Type name} = & \text{Cons}_1 \ t_{11} \dots t_{1k_1} \\ & | \text{Cons}_2 \ t_{21} \dots t_{2k_2} \\ & | \dots \\ & | \text{Cons}_n \ t_{n1} \dots t_{nk_n} \end{aligned}$$

unde $k_1, \dots, k_n \geq 0$

Atenție! Alternativele trebuie să conțină **constructori**!

data StrInt = **String** | **Int** este **greșit**.

data StrInt = VS **String** | VI **Int** este corect.

[VI 1, VS "abc", VI 34, VI 0, VS "xyz"] :: [StrInt]

Tipuri de date algebrice - exemple

```
data Bool = False | True
```

```
data Season = Winter | Spring | Summer | Fall
```

```
data Shape = Circle Float | Rectangle Float Float
```

```
data Pair a b = Pair a b
```

```
-- constructorul de tip și cel de date pot să coincidă
```

```
data Exp = Lit Int | Add Exp Exp | Mul Exp Exp
```

```
data List a = Nil | Cons a (List a)
```

```
data Tree a = Empty | Leaf a | Branch (Tree a) (Tree a)
```

Exemplu: numerele naturale (Peano)

Declarație ca tip de date algebric

```
data Nat = Zero | Succ Nat
```

Adunarea:

```
(+++)  
m +++ Zero      = m  
m +++ (Succ n) = Succ (m +++ n)
```

Înmulțirea:

```
(***)  
m *** Zero      = Zero  
m *** (Succ n) = (m *** n) +++ m
```

Quiz time!



<https://tinyurl.com/PF-C05-Quiz1>

Tipul Maybe

```
data Maybe a = Nothing | Just a  
  
divide :: Int -> Int -> Maybe Int  
divide n 0 = Nothing  
divide n m = Just (n 'div' m)
```

Tipul Maybe

```
data Maybe a = Nothing | Just a
```

```
divide :: Int -> Int -> Maybe Int
```

```
divide n 0 = Nothing
```

```
divide n m = Just (n 'div' m)
```

```
safeHead :: [a] -> Maybe a
```

```
safeHead [] = Nothing
```

```
safeHead (x:xs) = Just x
```

Tipul Maybe

```
divide :: Int -> Int -> Maybe Int
divide n 0 = Nothing
divide n m = Just (n 'div' m)
```

```
-- utilizare gresita
wrong :: Int -> Int -> Int
wrong n m = divide n m + 3
```

```
-- utilizare corecta
right :: Int -> Int -> Int
right n m = case divide n m of
    Nothing -> 3
    Just r   -> r + 3
```


Tipul Either

```
data Either a b = Left a | Right b
```

```
mylist :: [Either Int String]
```

```
mylist = [Left 4, Left 1, Right "hello", Left 2,  
          Right "␣", Right "world", Left 17]
```

O funcție care calculează suma elementelor întregi.

```
addints    :: [Either Int String] -> Int
```

```
addints    []                = 0
```

```
addints    (Left n : xs)     = n + addints xs
```

```
addints    (Right s : xs)    = addints xs
```

```
addints'   :: [Either Int String] -> Int
```

```
addints'   xs = sum [n | Left n <- xs]
```

Tipul Either

```
data Either a b = Left a | Right b
```

```
mylist :: [Either Int String]
```

```
mylist = [Left 4, Left 1, Right "hello", Left 2,  
          Right "␣", Right "world", Left 17]
```

O funcție care întoarce concatenarea elementelor de tip **String**.

```
addstrs    :: [Either Int String] -> String
```

```
addstrs    []                = ""
```

```
addstrs    (Left n : xs)     = addstrs xs
```

```
addstrs    (Right s : xs)    = s ++ addstrs xs
```

```
addstrs'   :: [Either Int String] -> String
```

```
addstrs'   xs = concat [s | Right s <- xs]
```

Cu **type** se pot redenumi tipuri deja existente.

```
type FirstName = String
```

```
type LastName  = String
```

```
type Age       = Int
```

```
type Height    = Float
```

```
type Phone     = String
```

```
data Person = Person FirstName LastName Age Height Phone
```

Proiecții

```
data Person = Person FirstName LastName Age Height Phone
```

```
firstName :: Person -> String
```

```
firstName (Person firstname _ _ _ _) = firstname
```

```
lastName :: Person -> String
```

```
lastName (Person _ lastname _ _ _ _) = lastname
```

```
age :: Person -> Int
```

```
age (Person _ _ age _ _) = age
```

```
height :: Person -> Float
```

```
height (Person _ _ _ height _) = height
```

```
phoneNumber :: Person -> String
```

```
phoneNumber (Person _ _ _ _ number) = number
```

```
data Person = Person { firstName :: String  
                      , lastName :: String  
                      , age :: Int  
                      , height :: Float  
                      , phoneNumber :: String  
                      }
```

```
gigel = Person { firstName = "Gheorghe"  
                , lastName="Georgescu"  
                , age = 30, height = 192.3  
                , phoneNumber = "0798765432"  
                }
```

- Putem folosi și pattern-matching
- Proiecțiile sunt definite automat

```
firstName :: Person -> String  
firstName gigel = "Gheorghe"
```

- Sintaxă specializată pentru actualizări

```
nextYear :: Person -> Person  
nextYear person = person { age = age person + 1 }
```

Înregistrări

```
data Person = Person { firstName :: String
                      , lastName  :: String
                      , age       :: Int
                      , height    :: Float
                      , phoneNumber :: String
                      }

ionel = Person "Ion" "Ionescu" 20 175.2 "0712334567"

nextYear person = person { age = age person + 1 }
```

```
Prelude> nextYear ionel
No instance for (Show Person) arising from a use of
  'print'
```

Deși toate definițiile sunt corecte, o valoare de tip `Person` nu poate fi afișată deoarece nu are o instanță a clasei **Show**.

Quiz time!



<https://tinyurl.com/PF-C05-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C06

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Clase de tipuri

Exemplu: test de apartenență

Problemă. Să definim o funcție care testează dacă un element aparține unei liste.

- folosind recursivitate

```
myElem x []      = False  
myElem x (y:ys) = x == y || myElem x ys
```

- folosind descrieri de liste

```
myElem x ys      = or [ x == y | y <- ys ]
```

- folosind funcții de nivel înalt

```
myElem x ys      = foldr (||) False (map (x ==) ys)
```

Funcția este polimorfică

Care este tipul funcției myElem?

Funcția myElem este **polimorfică**.

Definiția funcției este parametrică în tipul de date.

```
Prelude> myElem 1 [2,3,4]
```

```
False
```

```
Prelude> myElem 'o' "word"
```

```
True
```

```
Prelude> myElem (1,'o') [(0,'w'),(1,'o'),(2,'r')]
```

```
True
```

```
Prelude> myElem "word" ["list","of","word"]
```

```
True
```

Funcția este polimorfică ... dar nu pentru orice tip

Totuși myElem nu funcționează pentru orice tip!

```
Prelude> myElem (+ 2) [(+ 2), sqrt]
```

```
No instance for (Eq (Double -> Double)) arising from a  
  use of 'elem'
```

Ce se întâmplă?

```
myElem x ys = or [ x == y | y <- ys ]
```

```
Prelude> :t myElem
```

```
my_elem :: Eq a => a -> [a] -> Bool
```

Folosim relația de egalitate == care nu este definită pentru orice tip.

```
Prelude> ("ab",1) == ("ab",2)
```

```
False
```

```
Prelude> sqrt == sqrt
```

```
No instance for (Eq (Double -> Double)) ...
```

Informal, o **clasă de tipuri** este o mulțime de funcții.

Puteți gândi o clasă de tipuri ca o interfață.

Puteți verifica ce conține o anumită clasă de tipuri folosind comanda **:info** sau **:i**.

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
  -- minimum definition: (==)
  x /= y = not (x == y)
  -- ^^ putem avea definitii implicite
```

Tipurile care aparțin clasei sunt **instanțe** ale clasei.

```
instance Eq Bool where
  False == False = True
  False == True  = False
  True   == False = False
  True   == True  = True
```


Clasa de tipuri. Constrângeri de tip

În semnatura funcției `myElem` trebuie să precizăm ca tipul `a` este în clasa `Eq`

```
myElem :: Eq a => a -> [a] -> Bool
```

- `Eq a` se numește **constrângere de tip**.
- `=>` separă constrângerile de tip de restul semnaturii.

Clasa de tipuri. Constrângeri de tip

În exemplul anterior `myElem` este definită pe liste, dar funcția poate fi mai complexă:

```
Prelude> :t myElem  
myElem :: (Eq a, Foldable t) => a -> t a -> Bool
```

În această definiție `Foldable` este o altă clasă de tipuri, iar `t` este un parametru care ține locul unui *constructor de tip*!

Sistemul tipurilor in Haskell este complex!

Instance ale clasei Eq

```
class Eq a where
```

```
    (==) :: a -> a -> Bool
```

```
instance Eq Int where
```

```
    (==) = eqInt    -- built-in
```

```
instance Eq Char where
```

```
    x == y = ord x == ord y
```

```
instance (Eq a, Eq b) => Eq (a,b) where
```

```
    (u,v) == (x,y) = (u == x) && (v == y)
```

```
instance Eq a => Eq [a] where
```

```
    [] == []      = True
```

```
    [] == y:ys    = False
```

```
    x:xs == []    = False
```

```
    x:xs == y:ys = (x == y) && (xs == ys)
```

Clasele pot fi extinse:

```
class (Eq a) => Ord a where
  (<)  :: a -> a -> Bool
  (<=) :: a -> a -> Bool
  (>)  :: a -> a -> Bool
  (>=) :: a -> a -> Bool
  -- minimum definition: (<=)
  x < y    = x <= y && x /= y
  x > y    = y < x
  x >= y   = y <= x
```

Clasa **Ord** este clasa tipurilor de date cu o relație de ordine.

În definiția clasei **Ord** s-a impus o constrângere de tip.

Orice instanță a clasei **Ord** trebuie să fie și instanță a clasei **Eq**.

Instanțe ale clasei Ord

```
instance Ord Bool where
    False <= False = True
    False <= True  = True
    True  <= False = False
    True  <= True  = True
```

```
instance (Ord a, Ord b) => Ord (a,b) where
    (x,y) <= (x',y') = x < x' || (x == x' && y <= y')
    -- ordinea lexicografica
```

```
instance Ord a => Ord [a] where
    []      <= ys      = True
    (x:xs) <= []      = False
    (x:xs) <= (y:ys) = x < y || (x == y && xs <= ys)
```

Definirea claselor

Să presupunem că vrem să definim o clasă de tipuri pentru datele care pot fi afișate. O astfel de clasă trebuie aibă o funcție care să indice modul de afișare:

```
class Visible a where  
    toString :: a -> String
```

Putem face instanțieri astfel:

```
instance Visible Char where  
    toString c = [c]
```

Clasele Eq și Ord sunt predefinite.

Clasa Visible este definită de noi.

Există o clasă predefinită care are același rol: clasa **Show**.

Clasa Show

```
class Show a where  
  show :: a -> String    -- analogul lui "toString"
```

```
instance Show Bool where  
  show False      = "False"  
  show True       = "True"
```

```
instance (Show a, Show b) => Show (a,b) where  
  show (x,y) = "(" ++ show x ++ "," ++ show y ++ ")"
```

```
instance Show a => Show [a] where  
  show []      = "[]"  
  show (x:xs) = "[" ++ showSep x xs ++ "]"  
  where  
    showSep x []      = show x  
    showSep x (y:ys) = show x ++ "," ++ showSep y ys
```

Clase de tipuri pentru numere

```
class (Eq a, Show a) => Num a where
  (+), (-), (*)      :: a -> a -> a
  negate             :: a -> a
  ...
  fromInteger        :: Integer -> a
  -- minimum definition: (+), (-), (*), fromInteger
  negate x           = fromInteger 0 - x
```

```
class (Num a) => Fractional a where
  (/)                :: a -> a -> a
  recip              :: a -> a
  fromRational        :: Rational -> a
  ...
  -- minimum definition: (/), fromRational
  recip x             = 1/x
```


Derivare automata pentru tipuri algebrice

Să presupunem că avem tipurile de date:

```
data Season = Spring | Summer | Autumn | Winter
```

```
data Point a b = Pt a b
```

Cum putem să le facem instanțe ale claselor **Eq**, **Ord**, **Show**?

Derivare automata pentru tipuri algebrice

Putem să le facem explicit sau să folosim derivarea automată.

```
data Season = Spring | Summer | Autumn | Winter  
           deriving (Eq, Ord, Show)
```

```
data Point a b = Pt a b  
           deriving (Eq, Ord, Show)
```

Atenție! Derivarea automată poate fi folosită numai pentru unele clase predefinite.

Derivare automata vs Instanțiere explicită

Instanțierea prin derivare automată:

```
data Point a b = Pt a b  
           deriving Eq
```

Instanțiere explicită:

```
instance Eq a => Eq (Point a b) where  
    (==) (Pt x1 y1) (Pt x2 y2) = (x1 == x2)
```

Derivare automata pentru tipuri algebrice

```
data Point a b = Pt a b
           deriving (Eq, Ord, Show)
```

Egalitatea, relația de ordine și modalitatea de afișare sunt definite implicit dacă este posibil:

```
Prelude> Pt 2 3 < Pt 5 6
True
```

```
Prelude> Pt 2 "b" < Pt 2 "a"
False
```

```
Prelude> Pt (+2) 3 < Pt (+5) 6
```

```
No instance for (Ord (Integer -> Integer)) arising from
a use of '<'
```

Quiz Time!



<https://tinyurl.com/PF-C06-Quiz1>

Definirea claselor

```
class Size a where  
  size :: a -> Int
```

```
instance Size Int where  
  size x = abs x
```

```
instance Size [a] where  
  size xs = length xs
```

Definirea claselor

```
Prelude> :t size  
size :: Size a => a -> Int
```

```
Prelude> size [True,False]  
2
```

```
Prelude> sizeBoth a b = [size a, size b]
```

```
Prelude> :t sizeBoth  
sizeBoth :: (Size a1, Size a2) => a1 -> a2 -> [Int]
```

Definirea claselor

O clasă poate conține mai multe funcții și chiar constante.

De exemplu, putem avea aceasta versiune a clasei `Size`:

```
class Size a where
  empty :: a
  size :: a -> Int
  sameSize :: a -> a -> Bool

instance Size (Maybe a) where
  empty = Nothing

  size Nothing = 0
  size (Just a) = 1

  sameSize x y = size x == size y
```



```
class Size a where  
  empty :: a  
  size :: a -> Int  
  sameSize :: a -> a -> Bool  
  
instance Size [a] where  
  empty = []  
  size xs = length xs  
  sameSize x y = size x == size y
```

Implementări implicite

```
class Example a where
```

```
  example :: a          -- exemplul de baza pentru tipul a  
  examples :: [a]       -- o lista scurte de exemple  
  examples = [example]  -- implicit
```

```
instance Example Int where
```

```
  example = 1  
  examples = [0,1,2]
```

```
instance Example Bool where
```

```
  example = True
```

Implementări implicite

```
Prelude> example :: Bool  
True
```

```
Prelude> example :: Int  
1
```

```
Prelude> examples :: [Bool]  
[True]
```

```
Prelude> examples :: [Int]  
[0,1,2]
```

Mini Test!



<https://tinyurl.com/PF-MiniTest>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C07

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Functori

În cursurile trecute, am văzut funcția

```
map :: (a -> b) -> [a] -> [b]
```

Problemă. Putem generaliza această funcție la alte tipuri parametrizate?


```
class Functor f where  
  fmap :: (a -> b) -> f a -> f b
```

Pentru o funcție $g :: a \rightarrow b$ și o valoare $ca :: f\ a$,
fmap produce $cb :: f\ b$,
obținută prin transformarea lui ca folosind funcția g (și doar atât!).

Instanță pentru liste

```
instance Functor [] where  
  fmap = map
```

```
Prelude> fmap (*2) [1..3]  
[2,4,6]
```

```
Prelude> map (*2) [1..3]  
[2,4,6]
```

```
Prelude> fmap (++"1") ["1", "2", "3"]  
["11", "21", "31"]
```

Instanță pentru Maybe

```
class Functor f where  
  fmap :: (a -> b) -> f a -> f b
```

Instanță pentru Maybe

```
fmap :: (a -> b) -> Maybe a -> Maybe b
```

```
instance Functor Maybe where  
  fmap _ Nothing = Nothing  
  fmap f (Just x) = Just (f x)
```

Exemple

```
Prelude> fmap (*2) (Just 200)  
Just 400
```

```
Prelude> fmap (*2) Nothing  
Nothing
```

```
Prelude> fmap (++ "_world!") (Just "Hello")  
Just "Hello_world!"
```

Instanță pentru Either

```
class Functor f where  
  fmap :: (a -> b) -> f a -> f b
```

Instanță pentru Either e

```
fmap :: (a -> b) -> Either e a -> Either e b
```

```
instance Functor (Either e) where  
  fmap _ (Left x) = Left x  
  fmap f (Right y) = Right (f y)
```

Exemple

```
Prelude> fmap (*2) (Right 6)  
Right 12  
Prelude> fmap (*2) (Left 135)  
Left 135
```

Instanță pentru Tree

```
class Functor f where  
    fmap :: (a -> b) -> f a -> f b
```

Instanță pentru Tree

```
data Tree a = Empty  
           | Node a (Tree a) (Tree a)
```

```
fmap :: (a -> b) -> Tree a -> Tree b
```

```
instance Functor Tree where  
    fmap f Empty = Empty  
    fmap f (Node x l r) = Node (f x) (fmap f l) (fmap f r)
```

Instanță pentru tipul funcție

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

Instanță pentru tipul funcție

Tipul funcțiilor de sursă dată parametric în tipul a este $t \rightarrow a$.

```
fmap :: (a -> b) -> (t -> a) -> (t -> b)
```

```
instance Functor (->) t where
  fmap f g = f . g      -- sau, mai simplu, fmap = (.)
```

Exemple

```
Prelude> fmap (*2) (+100) 4
208
```

```
Prelude> (fmap . fmap) (+1) [Just 1, Just 2, Just 3]
[Just 2, Just 3, Just 4]
```

fmap ca operator

```
fmap :: Functor f => (a -> b) -> f a -> f b  
(<$>) :: Functor f => (a -> b) -> f a -> f b
```

Exemple

```
(+1) <$> [1,2,3]      ==> [2,3,4]  
not <$> Just False    ==> Just True
```

```
reverse . tail $      "hello"      ==> "olle"  
reverse . tail <$> Just "hello"     ==> Just "olle"  
-- echivalent cu  
fmap (reverse . tail) (Just "hello") ==> Just "olle"
```

Proprietăți ale functorilor

- Argumentul `f` al lui `Functor f` definește o transformare de tipuri
 - `f a` este tipul `a` transformat prin functorul `f`
- `fmap` definește transformarea corespunzătoare a funcțiilor
 - `fmap :: (a -> b) -> (f a -> f b)`

Contractul lui `fmap`

- `fmap g ca` e obținută prin transformarea rezultatelor produse de computația `ca` folosind funcția `g` (și doar atât!)
- Abstractizat prin două legi:
 - identitate** `fmap id == id`
 - compunere** `fmap (g . h) == fmap g . fmap h`

Invalidarea contractului - identitate

```
data WhoCares a = ItDoesnt
                | Matter a
                | WhatThisIsCalled
                deriving (Eq, Show)
```

Instanță a clasei Functor care invalidează condiția de conservare a identității:

```
instance Functor WhoCares where
    fmap _ ItDoesnt = WhatThisIsCalled
    fmap _ WhatThisIsCalled = ItDoesnt
    fmap f (Matter a) = Matter (f a)
```

```
Prelude> fmap id ItDoesnt
WhatThisIsCalled
Prelude> id ItDoesnt
ItDoesnt
```

Validarea contractului - identitate

```
data WhoCares a = ItDoesnt
                | Matter a
                | WhatThisIsCalled
                deriving (Eq, Show)
```

Instanță a clasei Functor care validează condiția de conservare a identității:

```
instance Functor WhoCares where
    fmap _ ItDoesnt = ItDoesnt
    fmap _ WhatThisIsCalled = WhatThisIsCalled
    fmap f (Matter a) = Matter (f a)
```

```
Prelude> fmap id ItDoesnt
ItDoesnt
Prelude> id ItDoesnt
ItDoesnt
```

Invalidarea contractului - compunere

```
data BigKnockTheory a =  
  Sheldon Int a  
  deriving (Eq, Show)
```

Instanță a clasei Functor care invalidează condiția de conservare a compunerii:

```
instance Functor BigKnockTheory where  
  fmap f (Sheldon n a) = Sheldon (n+1) (f a)
```

```
Prelude> howManyPauses = Sheldon 1 "knock_knock"
```

```
Prelude> f = (++ "_Penny")
```

```
Prelude> g = (++ "_knock")
```

```
Prelude> fmap (f . g) howManyPauses
```

```
Sheldon 2 "knock_knock_knock_Penny"
```

```
Prelude> fmap f . fmap g $ howManyPauses
```

```
Sheldon 3 "knock_knock_knock_Penny"
```

Validarea contractului - compunere

```
data BigKnockTheory a =  
  Sheldon Int a  
  deriving (Eq, Show)
```

Instanță a clasei Functor care validează condiția de conservare a compunerii:

```
instance Functor BigKnockTheory where  
  fmap f (Sheldon n a) = Sheldon n (f a)
```

```
Prelude> howManyLaughs = Sheldon 0 "knock_knock"
```

```
Prelude> f = (++ "_Penny")
```

```
Prelude> g = (++ "_knock")
```

```
Prelude> fmap (f . g) howManyLaughs
```

```
Sheldon 0 "knock_knock_knock_Penny"
```

```
Prelude> fmap f . fmap g $ howManyLaughs
```

```
Sheldon 0 "knock_knock_knock_Penny"
```

Quiz Time!



<https://tinyurl.com/PF-C07-Quiz1>

Functori applicativi

Cum putem "concatena" **Just** "Hey" cu **Just** "You!"?

Vrem să obținem **Just** "Hey_You!"

Cum putem "concatena" **Just** "Hey" cu **Just** "You!"?

Vrem să obținem **Just** "Hey_You!"

```
Prelude> :t (++) <$> (Just "Hey_")  
(++) <$> (Just "Hey_") :: Maybe (String -> String)
```

```
Prelude> (++) <$> (Just "Hey_") <*> (Just "You!")  
Just "Hey_You!"
```


Problema

- Folosind **fmap** putem transforma o funcție **$h :: a \rightarrow b$** într-o funcție **$fmap\ h :: m\ a \rightarrow m\ b$**
- Dar ce se întâmplă dacă avem o funcție cu mai multe argumente?
De exemplu, cum trecem de la **$h :: a \rightarrow b \rightarrow c$** la **$h' :: m\ a \rightarrow m\ b \rightarrow m\ c$**
- Putem încerca să folosim **fmap**
- Dar, deoarece **$h :: a \rightarrow (b \rightarrow c)$** , obținem
 $fmap\ h :: m\ a \rightarrow m\ (b \rightarrow c)$
- Putem aplica **fmap h** la o valoare **$ca :: m\ a$** și obținem
 $fmap\ h\ ca :: m\ (b \rightarrow c)$

Cum transformăm un obiect din $m \ (b \rightarrow c)$ într-o funcție $m \ b \rightarrow m \ c$?

- $\mathbf{ap} :: m \ (b \rightarrow c) \rightarrow (m \ b \rightarrow m \ c)$, sau, ca operator
- $(\langle * \rangle) :: m \ (b \rightarrow c) \rightarrow m \ b \rightarrow m \ c$

Merge pentru funcții cu oricâte argumente!

Dată fiind o funcție

$$f :: a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_n \rightarrow a$$

și computațiile

$$ca_1 :: m\ a_1, ca_2 :: m\ a_2, \dots, ca_n :: m\ a_n,$$

vrem să „aplicăm” funcția f pe rând computațiilor $ca_1, \dots, ca_n$ pentru a obține o computație finală $ca :: m\ a$.

Cazul general

Date fiind

- $f :: a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_n \rightarrow a$
- $ca_1 :: m\ a_1, ca_2 :: m\ a_2, \dots, ca_n :: m\ a_n,$
- $fmap :: (a \rightarrow b) \rightarrow m\ a \rightarrow m\ b$
- $(<*>) :: m\ (b \rightarrow c) \rightarrow m\ b \rightarrow m\ c$ cu „proprietăți bune”

Atunci

$fmap\ f :: m\ a_1 \rightarrow m\ (a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_n \rightarrow a)$

$fmap\ f\ ca_1 :: m\ (a_2 \rightarrow a_3 \rightarrow \dots \rightarrow a_n \rightarrow a)$

$fmap\ f\ ca_1\ <*>\ ca_2 :: m\ (a_3 \rightarrow \dots \rightarrow a_n \rightarrow a)$

...

$fmap\ f\ ca_1\ <*>\ ca_2\ <*>\ ca_3\ \dots\ <*>\ ca_n :: m\ a$

Clasa de tipuri Applicative

```
class Functor m => Applicative m where  
  pure :: a -> m a  
  (<*>) :: m (a -> b) -> m a -> m b
```

- Orice instanță a lui `Applicative` trebuie să fie instanță a lui **Functor**
- `pure` transformă o valoare într-o computație minimală care are acea valoare ca rezultat, și nimic mai mult!
- `(<*>)` ia o computație care produce funcții și o computație care produce argumente pentru funcții și obține o computație care produce rezultatele aplicării funcțiilor asupra argumentelor

```
class Functor m where  
  fmap :: (a -> b) -> m a -> m b
```

```
class Functor m => Applicative m where  
  pure :: a -> m a  
  (<*>) :: m (a -> b) -> m a -> m b
```

Proprietate importantă

- `fmap f x == pure f <*> x`

Instanța pentru Maybe

```
class Functor m where
```

```
  fmap :: (a -> b) -> m a -> m b
```

```
class Functor m => Applicative m where
```

```
  pure :: a -> m a
```

```
  (<*>) :: m (a -> b) -> m a -> m b
```

```
instance Applicative Maybe where
```

```
  pure = Just
```

```
  Nothing <*> _ = Nothing
```

```
  Just f <*> x = fmap f x
```

Instanța pentru Maybe

```
Prelude> pure "Hey" :: Maybe String  
Just "Hey"
```

Cum concatenăm **Just "Hey"** cu **Just "You!"**?

```
Prelude> (++) <$> (Just "Hey_") <*> (Just "You!")  
Just "Hey_You!"
```

```
(++) :: String -> (String -> String)  
Just "Hey_" :: Maybe String  
(<$>) :: (a -> b) -> m a -> m b  
(++) <$> (Just "Hey_") :: Maybe (String -> String)  
Just "You!" :: Maybe String  
(<*>) :: m (a -> b) -> m a -> m b  
Just "Hey_You!" :: Maybe String
```


Instanța pentru Maybe

```
mDiv x y = if y == 0 then Nothing  
           else Just (x 'div' y)
```

```
mF x = (+) <$> pure 4 <*> mDiv 10 x
```

```
Prelude> mF 2  
Just 9
```

```
(+) :: Int -> Int -> Int  
pure 4 :: Maybe Int  
(<$>) :: (a -> b) -> m a -> m b  
(+) <$> pure 4 :: Maybe (Int -> Int)  
mDiv :: Int -> Int -> Maybe Int  
mDiv 10 x :: Maybe Int  
(<*>) :: m (b -> c) -> m b -> m c
```

Instanța pentru Either

```
class Functor m where
  fmap :: (a -> b) -> m a -> m b

class Functor m => Applicative m where
  pure :: a -> m a
  (<*>) :: m (a -> b) -> m a -> m b

instance Applicative (Either a) where
  pure = Right
  Left e  <*> _ = Left e
  Right f <*> x = fmap f x
```

```
Prelude> pure "Hey" :: Either a String  
Right "Hey"
```

```
Prelude> (++) <$> (Right "Hey_") <*> (Right "You!")  
Right "Hey_You!"
```

```
(++) :: String -> (String -> String)  
Right "Hey_" :: Either a String  
(++) <$> (Right "Hey_") :: Either a (String -> String)  
Right "You!" :: Either a String  
Right "Hey_You!" :: Either a String
```

Instanța pentru Either

```
eDiv x y = if y == 0 then Left "Division_by_0!"  
           else Right (x 'div' y)
```

```
eF x = (+) <$> pure 4 <*> eDiv 10 x
```

```
Prelude> eF 2  
Right 9
```

```
(+) :: Int -> Int -> Int  
pure 4 :: Either String Int  
(+) <$> pure 4 :: Either String (Int -> Int)  
eDiv :: Int -> Int -> Either String Int  
eDiv 10 x :: Either String Int  

```

```
class Functor m where  
    fmap :: (a -> b) -> m a -> m b  
  
class Functor m => Applicative m where  
    pure :: a -> m a  
    (<*>) :: m (a -> b) -> m a -> m b  
  
instance Applicative [] where  
    pure x = [x]  
    fs <*> xs = [f x | f <- fs, x <- xs]
```

Instanța pentru liste

```
Prelude> pure "Hey" :: [String]
["Hey"]
```

```
Prelude> (++) <$> ["Hello_", "Goodbye_"]
          <*> ["world", "happiness"]
["Hello_world", "Hello_happiness", "Goodbye_world", "
  Goodbye_happiness"]
```

```
(++) :: String -> (String -> String)
["Hello_", "Goodbye_"] :: [String]
(<$>) :: (a -> b) -> m a -> m b
(++) <$> ["Hello_", "Goodbye_"] :: [String -> String]
["world", "happiness"] :: [String]
(<*>) :: m (b -> c) -> m b -> m c
```

```
Prelude> (+) <$> [1,2] <*> [3,4]  
[4,5,5,6]
```

```
(+) :: Int -> Int -> Int
```

```
[1,2] :: [Int]
```

```
(<$>) :: (a -> b) -> m a -> m b
```

```
(<*>) :: m (b -> c) -> m b -> m c
```

```
Prelude> [(+), (*)] <*> [1,2] <*> [3,4]  
[4,5,5,6,3,4,6,8]
```

```
(+) , (*) :: Int -> Int -> Int  
[(+) , (*)] :: [Int -> Int -> Int]  
[1,2] :: [Int]  
(<*>) :: m (b -> c) -> m b -> m c  
[(+) , (*)] <*> [1,2] :: [Int -> Int]  
[(+) , (*)] <*> [1,2] <*> [3,4] :: [Int]
```


Recap

$$\begin{aligned}(\$) &:: (a \rightarrow b) \rightarrow a \rightarrow b \\(<\$>) &:: (a \rightarrow b) \rightarrow m\ a \rightarrow m\ b \\(<*>) &:: m\ (a \rightarrow b) \rightarrow m\ a \rightarrow m\ b\end{aligned}$$

Quiz Time!



<https://tinyurl.com/PF-C07-Quiz2>

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C08

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Monade - privire de ansamblu

Despre intenție și acțiune

[1] S. Peyton-Jones, *Tackling the Awkward Squad*: ...

- [1] A purely functional program implements a function; it has no side effect.
- [1] Yet the ultimate purpose of running a program is invariably to cause some side effect: a changed file, some new pixels on the screen, a message sent, ...

Exemplu

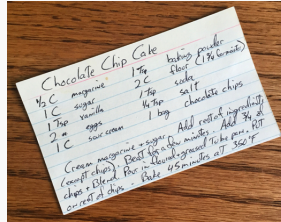
```
putChar :: Char -> IO ()  
Prelude> putChar '!'
```

reprezintă o comandă care, **dacă va fi executată**, va afișa un semn de exclamare.

Mind-Body Problem - Rețetă vs Prăjitură



c :: Cake



r :: Recipe Cake

IO este o rețetă care produce o valoare de tip **a**.

Motorul care citește și execută instrucțiunile **IO** se numește **Haskell Runtime System** (RTS). Acest sistem reprezintă legătura dintre programul scris și mediul în care va fi executat, împreună cu toate efectele și particularitățile acestuia.

Ce este o monadă?

Există multe răspunsuri, variind între

- O monadă este o clasă de tipuri în Haskell.
- "All told, a monad in X is just a monoid in the category of endofunctors in X, with product $\times$ replaced by composition of endofunctors and unit set by the identity endofunctor."

Saunders Mac Lane, Categories for the Working Mathematician, 1998.

- O monadă este un burrito. <https://byorgey.wordpress.com/2009/01/12/abstraction-intuition-and-the-monad-tutorial-fallacy/>



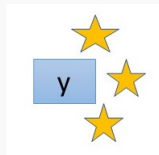
<https://twitter.com/monadburritos>

Funcții îmbogățite și efecte

- Funcție simplă: $x \mapsto y$

știind x , obținem **direct** y

- Funcție îmbogățită: $x \mapsto$



știind x , putem să **extragem** y și să producem un **efect**

Referințe:

<https://bartoszmilewski.com/2016/11/21/monads-programmers-definition/>

<https://bartoszmilewski.com/2016/11/30/monads-and-effects/>

Funcții îmbogățite și efecte

Funcție îmbogățită: $x \mapsto$



Exemplu:

Folosind tipul **Maybe a**

```
data Maybe a = Nothing | Just a
```

```
f :: Int -> Maybe Int
```

```
f x = if x < 0 then Nothing else Just (x+1)
```

Cum putem aplica f de două ori peste 0 și să obținem Just 2?

Clasa de tipuri Monad

```
class Applicative m => Monad m where
    (>=)  :: m a -> (a -> m b) -> m b
    (>>)  :: m a -> m b -> m b
    return :: a -> m a
```

În Haskell, monada este o clasă de tipuri!

Clasa **Monad** este o extensie a clasei **Applicative**!

- $m\ a$ este tipul **comenzilor** care produc rezultate de tip a (și au efecte laterale)
- $a \rightarrow m\ b$ este tipul **continuărilor** / a funcțiilor cu efecte laterale
- $>=$ este operația de „secvențiere” a comenzilor

Funcții îmbogățite și efecte

Funcție îmbogățită: $x \mapsto$



$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

Exemplu:

Folosind tipul **Maybe a**

```
data Maybe a = Nothing | Just a
```

```
f :: Int -> Maybe Int
```

```
f x = if x < 0 then Nothing else (Just x+1)
```

Cum putem aplica f de două ori peste 0 și să obținem Just 2?

```
> f 0 >>= f
```

Asociativitate și element neutru

Operația `>=>` este asociativă și are element neutru `return`

- Element neutru (la dreapta):

$$(\text{return } x) \gg= g = g \ x$$

- Element neutru (la stânga):

$$x \gg= \text{return} = x$$

- Asociativitate:

$$(f \gg= g) \gg= h = f \gg= (\backslash x \rightarrow (g \ x \gg= h))$$

Un exemplu care citește și afișează un string

```
getLine >=> putStrLn
```

```
(>=>) :: m a -> (a -> m b) -> m b
```

```
getLine :: IO String
```

```
putStrLn :: String -> IO ()
```

De ce nu este suficient `fmap`?

Exemplu: `putStrLn <$> getLine`

```
<$> :: Functor f => (a -> b) -> f a -> f b
```

```
getLine  :: IO String
```

```
putStrLn :: String -> IO ()
```

```
-- in exemplul nostru, b devine IO ()
```

```
--                                [1] [2] [3]
```

```
putStrLn <$> getLine :: IO (IO ())
```

[1] **IO** din exterior reprezintă efectul pe care **getLine** trebuie să îl producă pentru a obține un **String** introdus de utilizator

[2] **IO** din interior reprezintă efectul care s-ar produce dacă **putStrLn** a fost evaluat

[3] **()** este tipul unitate întors de **putStrLn**

De ce nu este suficient `fmap`?

```
putStrLn <$> getLine :: IO (IO ())
```

Trebuie să unim efectele lui `getLine` și `putStrLn` într-un singur efect **IO!**

```
import Control.Monad (join)
join :: Monad m => m (m a) -> m a
join $ putStrLn <$> getLine
```


Notăția `do` pentru monade

`(>>=)` $:: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

`(>>)` $:: m\ a \rightarrow m\ b \rightarrow m\ b$

Notăția cu operatori	Notăția do
<code>e >>= \x -> rest</code>	<code>x <- e</code> <code>rest</code>
<code>e >>= _ -> rest</code>	<code>e</code> <code>rest</code>
<code>e >> rest</code>	<code>e</code> <code>rest</code>

`binding' :: IO ()`

`binding' =`

`getLine >>=`

`putStrLn`

`binding :: IO ()`

`binding = do`

`name <- getLine`

`putStrLn name`

Notăția do pentru monade

```
twoBinds' :: IO ()
twoBinds' =
    putStrLn "name_pls:" >>
    getLine >>=

    \name ->
        putStrLn "age_pls:" >>
        getLine >>=

    \age ->
        putStrLn ("y_helo_thar:_ "
            ++ name ++ "_who_is:_ "
            ++ age ++ "_years_old.")
```

Notăția do pentru monade

```
twoBinds :: IO ()  
twoBinds = do  
    putStrLn "name_pls:"  
    name <- getLine  
  
    putStrLn "age_pls:"  
    age <- getLine  
  
    putStrLn ("y_helo_thar:_"  
              ++ name ++ "_who_is:_"  
              ++ age ++ "_years_old.")
```

Notăția **do** pentru monade

$(\gg=) \quad :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) \quad :: m\ a \rightarrow m\ b \rightarrow m\ b$

Notăția cu operatori	Notăția do
$e \gg= \backslash x \rightarrow \text{rest}$	$x \leftarrow e$ rest
$e \gg= \backslash _ \rightarrow \text{rest}$	e rest
$e \gg \text{rest}$	e rest

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg e3$

devine

Notăția **do** pentru monade

$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) :: m\ a \rightarrow m\ b \rightarrow m\ b$

Notăția cu operatori	Notăția do
$e \gg= \backslash x \rightarrow \text{rest}$	$x \leftarrow e$ rest
$e \gg= \backslash _ \rightarrow \text{rest}$	e rest
$e \gg \text{rest}$	e rest

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg e3$

devine

do

$x1 \leftarrow e1$

$e2$

$e3$

Notăția do pentru monade

$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) :: m\ a \rightarrow m\ b \rightarrow m\ b$

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg= \backslash x2 \rightarrow e3 \gg= \backslash_ \rightarrow e4 \gg= \backslash x4 \rightarrow e5$

devine

Notăția do pentru monade

$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) :: m\ a \rightarrow m\ b \rightarrow m\ b$

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg= \backslash x2 \rightarrow e3 \gg= \backslash_ \rightarrow e4 \gg= \backslash x4 \rightarrow e5$

devine

do

$x1 \leftarrow e1$

$x2 \leftarrow e2$

$e3$

$x4 \leftarrow e4$

$e5$

Functor și Applicative definiți cu return și >=>

```
instance Monad M where
```

```
    return a = ...
```

```
    ma >=> k = ...
```

```
instance Applicative M where
```

```
    pure = return
```

```
    mf <*> ma = do
```

```
        f <- mf
```

```
        a <- ma
```

```
        return (f a)
```

```
    -- mf >=> (\f -> ma >=> (\a -> return (f a)))
```

```
instance Functor M where
```

```
    fmap f ma = pure f <*> ma
```

```
    -- ma >=> \a -> return (f a)
```

```
    -- ma >=> (return . f)
```


Exemple de efecte laterale

I/O	Monada IO
Parțialitate	Monada Maybe
Excepții	Monada Either
Nedeterminism	Monada [] (listă)
Logging	Monada Writer
Stare	Monada State
Memorie read-only	Monada Reader

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C09

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Concurs "Best Cheat Sheet"

- cheat sheet-ul câștigător o să fie folosit de toată lumea la examen
- o foaie A4 față-verso
- participare individuală (nu în echipe)
- locul I primește un punct în plus la nota finală (dacă nota finală este peste 5) și recunoștința colegilor
- propunerile trebuie trimise în format pdf cu numele fișierului de forma

NUME_PRENUME_GRUPA.pdf

- propunerile se pot trimite până pe 9 ianuarie la acest link:
<https://tinyurl.com/cheatsheetpf2025>

Monade - recap

Functor, Applicative, Monad

```
class Functor f where
```

```
  fmap :: (a -> b) -> f a -> f b
```

```
class Functor m => Applicative m where
```

```
  pure :: a -> m a
```

```
  (<*>) :: m (a -> b) -> m a -> m b
```

```
class Applicative m => Monad m where
```

```
  (>=) :: m a -> (a -> m b) -> m b
```

```
  (>>) :: m a -> m b -> m b
```

```
  return :: a -> m a
```

```
ma >> mb = ma >= \_ -> mb
```

Notăția **do** pentru monade

$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) :: m\ a \rightarrow m\ b \rightarrow m\ b$

Notăția cu operatori	Notăția do
$e \gg= \backslash x \rightarrow \text{rest}$	$x \leftarrow e$ rest
$e \gg= \backslash _ \rightarrow \text{rest}$	e rest
$e \gg \text{rest}$	e rest

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg e3$

devine

do

$x1 \leftarrow e1$

$e2$

$e3$

Exemple de efecte laterale

I/O	Monada IO
Parțialitate	Monada Maybe
Excepții	Monada Either
Nedeterminism	Monada [] (listă)
Logging	Monada Writer
Stare	Monada State
Memorie read-only	Monada Reader

Monada List

Monada Listă (nedeterminism)

O computație care întoarce un rezultat nedeterminist nu este o funcție pură.

Poate fi transformată într-o funcție pură transformând rezultatul său din tipul `a` în tipul `[a]`.

În esență, construim o funcție care returnează toate rezultatele posibile în același timp.

Monada Listă (nedeterminism)

Funcțiile din clasa **Monad** specializate pentru liste:

```
(>=>)  :: [a] -> (a -> [b]) -> [b]  
return :: a -> [a]
```

```
instance Monad [] where  
  return x = [x]  
  xs >=> f = concat (map f xs)
```

```
concat :: [[a]] -> [a]
```

Monada Listă – exemplu

```
twiceWhenEven :: [Integer] -> [Integer]
twiceWhenEven xs = do
    x <- xs
    if even x
        then [x*x, x*x]
        else [x*x]
```

Monada Maybe

Monada Maybe (a funcțiilor parțiale)

```
data Maybe a = Nothing | Just a
```

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b  
return :: a -> Maybe a
```

```
instance Monad Maybe where  
    return = Just
```

```
Just va >>= f  = f va  
Nothing >>= _  = Nothing
```

Monada Maybe – exemplu

```
radical :: Float -> Maybe Float
radical x
  | x >= 0 = return (sqrt x)
  | x < 0  = Nothing

-- a * x^2 + b * x + c = 0
solEq2 :: Float -> Float -> Float -> Maybe Float
solEq2 0 0 0 = return 0
solEq2 0 0 c = Nothing
solEq2 0 b c = return (negate c / b)
solEq2 a b c = do
  rDelta <- radical (b * b - 4 * a * c)
  return ((negate b + rDelta) / (2 * a))
```

Monada Maybe – exemplu

```
-- a * x^2 + b * x + c = 0
solEq2All :: Float -> Float -> Float -> Maybe [Float]
solEq2All 0 0 0 = return [0]
solEq2All 0 0 c = Nothing
solEq2All 0 b c = return [negate c / b]
solEq2All a b c = do
    rDelta <- radical (b * b - 4 * a * c)
    let s1 = (negate b + rDelta) / (2 * a)
    let s2 = (negate b - rDelta) / (2 * a)
    return [s1,s2]
```


Monada Either

Monada Either (a excepțiilor)

```
data Either err a = Left err | Right a
```

```
(>>=) :: Either err a -> (a -> Either err b) ->  
      Either err b
```

```
return :: a -> Either err a
```

```
instance Monad (Either err) where  
  return = Right
```

```
    Right va >>= f = f va
```

```
    err >>= _ = err
```

```
-- Left verr >>= _ = Left verr
```

Monada Either – exemplu

```
radical :: Float -> Either String Float
radical x
  | x >= 0 = return (sqrt x)
  | x < 0  = Left "radical: argument negativ"

-- a * x^2 + b * x + c = 0
solEq2 :: Float -> Float -> Float -> Either String Float
solEq2 0 0 0 = return 0
solEq2 0 0 c = Left "ecuatie: fara solutie"
solEq2 0 b c = return (negate c / b)
solEq2 a b c = do
  rDelta <- radical (b * b - 4 * a * c)
  return ((negate b + rDelta) / (2 * a))
```

Quiz Time!



<https://tinyurl.com/PF-C09-Quiz1>

Monada Writer

Monada Writer (variantă simplificată)

```
newtype Writer log a = Writer {runWriter :: (a, log)}  
-- a este parametru de tip
```

Funcțiile din clasa **Monad** specializate pentru **Writer**:

```
(>=>) :: Writer log a -> (a -> Writer log b) ->  
      Writer log b  
return :: a -> Writer log a
```

```
instance Monad (Writer String) where  
  return va = Writer (va, "")  
  ma >=> f = let (va, log1) = runWriter ma  
               (vb, log2) = runWriter (f va)  
               in Writer (vb, log1 ++ log2)
```

Monada Writer - exemplu

```
newtype Writer log a = Writer {runWriter :: (a, log)}
```

```
tell :: log -> Writer log ()
```

```
tell msg = Writer ((), msg)
```

```
logIncrement :: Int -> Writer String Int
```

```
logIncrement x = do
```

```
    tell ("increment:" ++ show x ++ "--")
```

```
    return (x + 1)
```

```
logIncrement2 :: Int -> Writer String Int
```

```
logIncrement2 x = do
```

```
    y <- logIncrement x
```

```
    logIncrement y
```

```
> runWriter (logIncrement2 13)
```

```
(15,"increment:13--increment:14")
```

Monada Reader

Monada Reader (stare nemodificabilă)

```
newtype Reader env a = Reader {runReader :: env -> a}  
-- runReader :: Reader env a -> env -> a
```

```
instance Monad (Reader env) where  
  return = Reader const  
  -- return x = Reader (\_ -> x)
```

```
ma >>= k = Reader f  
  where  
    f env = let va = runReader ma env  
             in runReader (k va) env
```

Monada Reader - exemplu

```
newtype Reader env a = Reader {runReader :: env -> a}  
-- runReader :: Reader env a -> env -> a
```

```
ask :: Reader env env  
ask = Reader id
```

```
tom :: Reader String String  
tom = do  
  env <- ask -- gives the environment (here a String)  
  return (env ++ "␣This␣is␣Tom.")
```

```
jerry :: Reader String String  
jerry = do  
  env <- ask  
  return (env ++ "␣This␣is␣Jerry.")
```

Monada Reader - exemplu

```
tomAndJerry :: Reader String String
tomAndJerry = do
    t <- tom
    j <- jerry
    return (t ++ "\n" ++ j)

runJerryRun :: String
runJerryRun = runReader tomAndJerry "Who_is_this?"
```

Pe săptămâna viitoare!

Programare funcțională

Introducere în programarea funcțională folosind Haskell
C10

Claudia Chiriță

Denisa Diaconescu

Departamentul de Informatică, FMI, UB

Monade - recap

Functor, Applicative, Monad

```
class Functor f where
```

```
  fmap :: (a -> b) -> f a -> f b
```

```
class Functor m => Applicative m where
```

```
  pure :: a -> m a
```

```
  (<*>) :: m (a -> b) -> m a -> m b
```

```
class Applicative m => Monad m where
```

```
  (>=)  :: m a -> (a -> m b) -> m b
```

```
  (>>)  :: m a -> m b -> m b
```

```
  return :: a -> m a
```

```
ma >> mb = ma >= \_ -> mb
```

Notăția **do** pentru monade

$(\gg=) :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$

$(\gg) :: m\ a \rightarrow m\ b \rightarrow m\ b$

Notăția cu operatori	Notăția do
$e \gg= \backslash x \rightarrow \text{rest}$	$x \leftarrow e$ rest
$e \gg= \backslash _ \rightarrow \text{rest}$	e rest
$e \gg \text{rest}$	e rest

De exemplu

$e1 \gg= \backslash x1 \rightarrow e2 \gg e3$

devine

do

$x1 \leftarrow e1$

$e2$

$e3$

Exemple de efecte laterale

I/O	Monada IO
Parțialitate	Monada Maybe
Excepții	Monada Either
Nedeterminism	Monada [] (listă)
Logging	Monada Writer
Stare	Monada State
Memorie read-only	Monada Reader

Monada List

Monada Listă (nedeterminism)

O computație care întoarce un rezultat nedeterminist nu este o funcție pură.

Poate fi transformată într-o funcție pură transformând rezultatul său din tipul `a` în tipul `[a]`.

În esență, construim o funcție care returnează toate rezultatele posibile în același timp.

Monada Listă (nedeterminism)

Funcțiile din clasa **Monad** specializate pentru liste:

```
(>=>)  :: [a] -> (a -> [b]) -> [b]  
return :: a -> [a]
```

```
instance Monad [] where  
  return x = [x]  
  xs >=> f = concat (map f xs)
```

```
concat :: [[a]] -> [a]
```

Monada Listă – exemplu

```
twiceWhenEven :: [Integer] -> [Integer]
twiceWhenEven xs = do
  x <- xs
  if even x
    then [x*x, x*x]
    else [x*x]
```

Monada Maybe

Monada Maybe (a funcțiilor parțiale)

```
data Maybe a = Nothing | Just a
```

```
(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b  
return :: a -> Maybe a
```

```
instance Monad Maybe where  
    return = Just
```

```
Just va >>= f  = f va  
Nothing >>= _  = Nothing
```

Monada Maybe – exemplu

```
radical :: Float -> Maybe Float
```

```
radical x
```

```
    | x >= 0 = return (sqrt x)
```

```
    | x < 0  = Nothing
```

```
-- a * x^2 + b * x + c = 0
```

```
solEq2 :: Float -> Float -> Float -> Maybe Float
```

```
solEq2 0 0 0 = return 0
```

```
solEq2 0 0 c = Nothing
```

```
solEq2 0 b c = return (negate c / b)
```

```
solEq2 a b c = do
```

```
    rDelta <- radical (b * b - 4 * a * c)
```

```
    return ((negate b + rDelta) / (2 * a))
```


Monada Maybe – exemplu

```
-- a * x^2 + b * x + c = 0
solEq2All :: Float -> Float -> Float -> Maybe [Float]
solEq2All 0 0 0 = return [0]
solEq2All 0 0 c = Nothing
solEq2All 0 b c = return [negate c / b]
solEq2All a b c = do
    rDelta <- radical (b * b - 4 * a * c)
    let s1 = (negate b + rDelta) / (2 * a)
    let s2 = (negate b - rDelta) / (2 * a)
    return [s1,s2]
```

Monada Either

Monada Either (a excepțiilor)

```
data Either err a = Left err | Right a
```

```
(>>=) :: Either err a -> (a -> Either err b) ->  
      Either err b
```

```
return :: a -> Either err a
```

```
instance Monad (Either err) where  
  return = Right
```

```
    Right va >>= f = f va
```

```
    err >>= _ = err
```

```
-- Left verr >>= _ = Left verr
```

Monada Either – exemplu

```
radical :: Float -> Either String Float
radical x
  | x >= 0 = return (sqrt x)
  | x < 0  = Left "radical: argument negativ"

-- a * x^2 + b * x + c = 0
solEq2 :: Float -> Float -> Float -> Either String Float
solEq2 0 0 0 = return 0
solEq2 0 0 c = Left "ecuatie: fara solutie"
solEq2 0 b c = return (negate c / b)
solEq2 a b c = do
  rDelta <- radical (b * b - 4 * a * c)
  return ((negate b + rDelta) / (2 * a))
```

Monada Writer

Monada Writer (variantă simplificată)

```
newtype Writer log a = Writer {runWriter :: (a, log)}  
-- a este parametru de tip
```

Funcțiile din clasa **Monad** specializate pentru **Writer**:

```
(>=>) :: Writer log a -> (a -> Writer log b) ->  
      Writer log b  
return :: a -> Writer log a
```

```
instance Monad (Writer String) where  
  return va = Writer (va, "")  
  ma >=> f = let (va, log1) = runWriter ma  
               (vb, log2) = runWriter (f va)  
               in Writer (vb, log1 ++ log2)
```

Monada Writer - exemplu

```
newtype Writer log a = Writer {runWriter :: (a, log)}
```

```
tell :: log -> Writer log ()
```

```
tell msg = Writer ((), msg)
```

```
logIncrement :: Int -> Writer String Int
```

```
logIncrement x = do
```

```
    tell ("increment:" ++ show x ++ "--")
```

```
    return (x + 1)
```

```
logIncrement2 :: Int -> Writer String Int
```

```
logIncrement2 x = do
```

```
    y <- logIncrement x
```

```
    logIncrement y
```

```
> runWriter (logIncrement2 13)
```

```
(15,"increment:13--increment:14")
```

Monada Reader

Monada Reader (stare nemodificabilă)

```
newtype Reader env a = Reader {runReader :: env -> a}  
-- runReader :: Reader env a -> env -> a
```

```
instance Monad (Reader env) where  
  return = Reader const  
  -- return x = Reader (\_ -> x)
```

```
ma >>= k = Reader f  
  where  
    f env = let va = runReader ma env  
             in runReader (k va) env
```

Monada Reader - exemplu

```
newtype Reader env a = Reader {runReader :: env -> a}  
-- runReader :: Reader env a -> env -> a
```

```
ask :: Reader env env  
ask = Reader id
```

```
tom :: Reader String String  
tom = do  
  env <- ask -- gives the environment (here a String)  
  return (env ++ "␣This␣is␣Tom.")
```

```
jerry :: Reader String String  
jerry = do  
  env <- ask  
  return (env ++ "␣This␣is␣Jerry.")
```

Monada Reader - exemplu

```
tomAndJerry :: Reader String String
tomAndJerry = do
    t <- tom
    j <- jerry
    return (t ++ "\n" ++ j)

runJerryRun :: String
runJerryRun = runReader tomAndJerry "Who_is_this?"
```

Se dă tipul de date

```
data Fuel a = Fuel {getFuel :: Integer -> Maybe (Integer  
    , a)}
```

Să se scrie instanța clasei **Monad** pentru tipul `Fuel`, ținând cont că în timpul computațiilor se consumă câte o unitate din resursă (`fuel`), iar computațiile pot fi efectuate cât timp resursa nu este 0.

Exemplu:

```
test :: Fuel Int
```

```
test = pure 0 >=> pure . (+1) >=> pure . (+1)
```

```
failEx = getFuel test 1      -- Nothing
```

```
sucEx = getFuel test 2      -- Just (0,2)
```

```
instance Functor Fuel where  
  fmap f (Fuel m) =  
    Fuel (\i -> case m i of  
      Nothing -> Nothing  
      Just (i', a) -> Just (i'  
        ', f a)  
    )
```

```
instance Applicative Fuel where
  pure x = Fuel (\i -> Just (i, x))
  (Fuel mf) <*> (Fuel af) =
    Fuel (\i -> case mf i of
      Nothing -> Nothing
      Just (i', f) ->
        if i' <= 0
        then Nothing
```

```
else case af (i' - 1) of
  Nothing -> Nothing
  Just (i'', a) ->
    if i'' <= 0
    then Nothing
    else Just (i'' - 2, f a)
)
```


Succes la examen!